Problem Statement - Harbor Master : MariaDB

Manual deployment of High Availability (HA) database architectures is complex, error-prone, and time-consuming. System administrators lack a unified tool to seamlessly orchestrate multi-version MariaDB clusters, validate configurations prior to deployment, and manage complex traffic routing (LVS) and hybrid replication topologies without manual command-line intervention.

Project Expectations (Scope of Work)

1. Unified User Interface (UI)

- Develop a UI that allows users to select specific MariaDB versions and target servers.
- Automate the installation of MariaDB services on remote nodes based on the user's selection.

2. Cluster Orchestration & Validation

- Implement a **Config Validator** that checks the required parameter compatibility before deployment.
- Automate the bootstrapping of a synchronous MariaDB (Galera) Cluster only after the config check passes.

3. Load Balancer Integration (LVS)

- Provision and configure **2 LVS (Linux Virtual Server) nodes**.
- Configure these nodes to route application traffic to the healthy nodes within the MariaDB cluster.

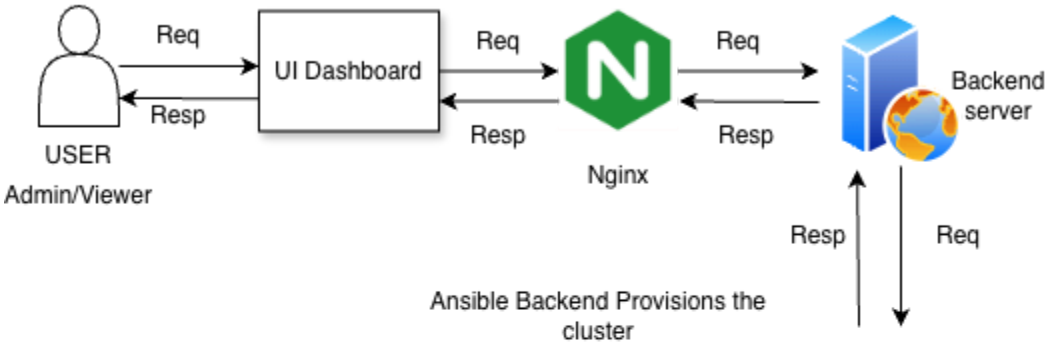
4. Hybrid Replication Setup

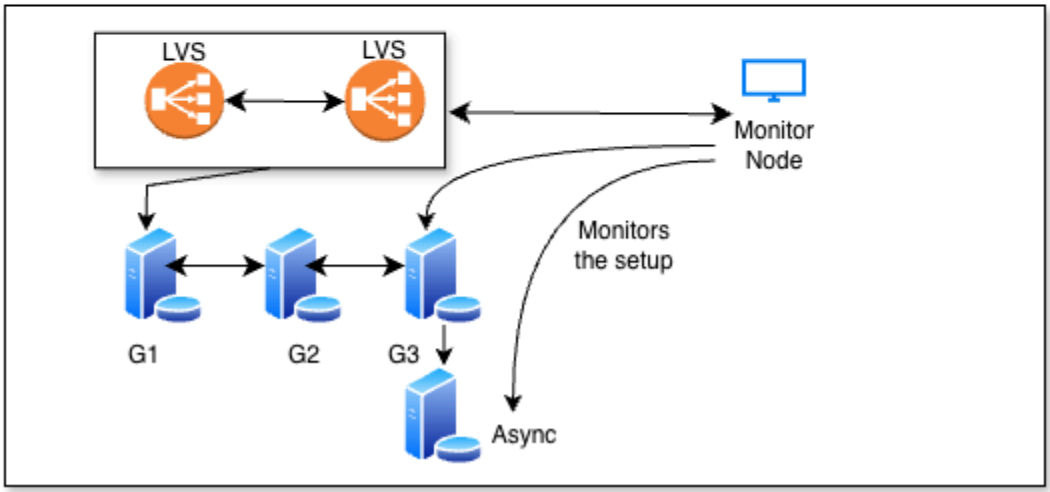
- Provision a standalone "Async Node" outside the cluster.

5. Integrated Monitoring Suite

- **System Level:** Monitor system-specific metrics.
- **Cluster Level:** Monitor database-specific metrics.

Req: Mariadb Setup Resp: Success/Failure & Live ansible logs





```
(venv) ubuntu@head-node:~/harbour-master$ cd backend/
(venv) ubuntu@head-node:~/harbour-master/backend$ tree -I 'venv'
.
├── Dockerfile
├── ansible
│   ├── deploy_cluster.yml
│   ├── inventory
│   │   ├── extravars.yml
│   │   └── hosts.yml
│   └── roles
│       ├── async_replica
│       │   ├── handlers
│       │   │   └── main.yml
│       │   ├── tasks
│       │   │   └── main.yml
│       │   └── templates
│       │       └── replica.cnf.j2
│       ├── common
│       │   └── tasks
│       │       └── main.yml
│       ├── galera
│       │   ├── defaults
│       │   │   └── main.yml
│       │   ├── handlers
│       │   │   └── main.yml
│       │   ├── tasks
│       │   │   └── main.yml
│       │   └── templates
│       │       ├── db_keepalived.conf.j2
│       │       ├── galera.cnf.j2
│       │       └── galera_hcip_manager.sh.j2
│       ├── lvs
│       │   ├── files
│       │   │   ├── id_rsa
│       │   │   └── id_rsa.pub
│       │   ├── handlers
│       │   │   └── main.yml
│       │   ├── tasks
│       │   │   └── main.yml
│       │   └── templates
│       │       └── keepalived.conf.j2
│       └── monitoring
│           ├── defaults
│           │   └── main.yml
│           ├── files
│           │   └── dashboards
│           │       ├── mariadb_deep_dive.json
│           │       └── system_metrics.json
│           ├── handlers
│           │   └── main.yml
│           ├── tasks
│           │   ├── exporters.yml
│           │   ├── main.yml
│           │   └── server.yml
│           └── templates
│               ├── dashboards_provider.yml.j2
│               ├── datasources.yml.j2
│               ├── galera_dashboard.json.j2
│               ├── grafana_alerts.yaml.j2
│               ├── grafana_contact_points.yaml.j2
│               ├── grafana_policies.yaml.j2
│               └── prometheus.yml.j2
├── api
│   ├── __pycache__
│   │   ├── health_checker.cpython-312.pyc
│   │   ├── main.cpython-312.pyc
│   │   └── schemas.cpython-312.pyc
│   ├── health_checker.py
│   ├── main.py
│   └── schemas.py
├── core
│   ├── __pycache__
│   │   ├── generator.cpython-312.pyc
│   │   ├── orchestrator.cpython-312.pyc
│   │   └── validator.cpython-312.pyc
│   ├── generator.py
│   ├── orchestrator.py
│   └── validator.py
├── env
│   └── extravars
└── requirements.txt
```

Implementation

Setting up the Infrastructure

On head-node: Sync Between Head Node and Managed Nodes

ubuntu@head-node: ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ansible

On local we add the public key of the head-node in a cloud-config.yml file and pass it in the launch of multipass

Step 1: multipass launch jammy --name db-01 --cpus 1 --memory 768M --disk 10G --cloud-init cloud-config.yaml

Step 2: multipass launch jammy --name db-02 --cpus 1 --memory 768M --disk 10G --cloud-init cloud-config.yaml

Step 3: multipass launch jammy --name db-03 --cpus 1 --memory 768M --disk 10G --cloud-init cloud-config.yaml

Step 4: multipass launch jammy --name lvs-01 --cpus 1 --memory 512M --disk 5G --cloud-init cloud-config.yaml

Step 5: multipass launch jammy --name lvs-02 --cpus 1 --memory 512M --disk 5G --cloud-init cloud-config.yaml

Step 6: multipass launch jammy --name async --cpus 1 --memory 512M --disk 10G --cloud-init cloud-config.yaml

Step 7: multipass launch jammy --name monitoring --cpus 1 --memory 768M --disk 10G --cloud-init cloud-config.yaml

ubuntu@head-node:~$ mkdir ~/harbor-master

ubuntu@head-node:~$ cd ~/harbor-master

ubuntu@head-node:~$ python3 -m venv venv

ubuntu@head-node:~$ source venv/bin/activate

ubuntu@head-node:~$ sudo apt install python3-pip , Python 3.12.3

ubuntu@head-node:~$ sudo apt update && sudo apt install -y python3-venv python3-full

(venv) ubuntu@head-node:~$ pip install --upgrade pip

(venv) ubuntu@head-node:~$ pip install fastapi uvicorn PyYAML pydantic paramiko ansible-runner

`fastapi`: A modern, high-performance web framework for building APIs with Python.
Uvicorn is a web server. It handles network communication - receiving requests from client applications such as users' browsers and sending responses to them.

`pydantic`: This handles data validation. It ensures that the information coming into your API (like an IP address or a username) is in the correct format.

`PyYAML`: Since Ansible heavily relies on `.yml` files, this library allows Python to read, write, and understand YAML configuration files

`paramiko`: paramiko allows your Python code to log into servers, run commands, and move files programmatically.
`ansible-runner`: This is a tool that allows you to run Ansible "playbooks" directly from inside a Python script.

(venv) ubuntu@head-node:~/harbor-master$ mkdir -p api core ansible/inventory ansible/playbooks ansible/roles/{common,galera,lvs,async_replica,monitoring} logs

(venv) ubuntu@head-node:~/harbor-master$ touch api/main.py api/schemas.py

(venv) ubuntu@head-node:~/harbor-master$ touch core/validator.py core/generator.py core/orchestrator.py
(venv) ubuntu@head-node:~/harbor-master$ touch core/validator.py core/generator.py core/orchestrator.py

(venv) ubuntu@head-node:~/harbor-master$ sudo apt install tree -y

(venv) ubuntu@head-node:~/harbor-master$ tree -I 'venv'

(venv) ubuntu@head-node:~/harbor-master$ pip install ansible

(venv) ubuntu@head-node-test:~/harbor-master/backend$ sudo cat requirements.txt

```
None
# --- Web Framework & Server ---
fastapi==0.109.0
uvicorn==0.27.0
python-multipart==0.0.6
# --- Data Validation ---
pydantic==2.5.3
PyYAML==6.0.1
# --- Ansible Integration ---
ansible-core==2.16.2
ansible-runner==2.3.4
# --- Networking & SSH ---
# (Using a modern version to support Ed25519 and avoid the q-bit error)
paramiko==3.4.0
cryptography==42.0.2
# --- Utilities ---
requests==2.31.0
psutil==5.9.8
watchfiles==0.21.0
pymysql==1.1.0
```

python-multipart: A library that allows FastAPI to parse and handle form data and file uploads from HTTP requests.
cryptography: A foundational package that provides low-level cryptographic recipes (like encryption and signing) used by Paramiko.
requests: The gold standard library for sending synchronous HTTP requests to interact with external APIs.
psutil: A cross-platform library for retrieving information on running processes and system utilization (CPU, memory, disks).
watchfiles: A simple and efficient tool for monitoring file system changes, often used for auto-reloading code during development.
pymysql: A pure-Python MySQL client library that allows your application to communicate with MySQL/MariaDB databases.

Schema Declaration

(venv) ubuntu@head-node:~/harbor-master$ sudo vim api/schemas.py

Summary of script -
Basemodel - turns json data into python objects
Fields - add metadata to specific attributes
Fields validator - a decorator to check a single field at a time
Model validator - a decorator to check multiple fields
Import re - regular expression validator
IP_REGREX - makes sure the ip i no is from 0 to 225 as 4 fields , and is valid
Fields(....) - this means that the field is required
Data integrity check
Required fields are present
Optional fields are provided default values
No two node name are same
SST:USER format is checked
All ips are unique
DB_admin password ! = sst_user password

```
None
```

```
from pydantic import BaseModel, Field, field_validator, model_validator
from typing import List, Optional
import re
IP_REGEX =
r"^(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(?:
25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$"
class GaleraNodeConfig(BaseModel):
    """Configuration for an individual Galera Cluster node."""
    ip: str = Field(..., description="IPv4 address of the node")
    node_name: str = Field(..., description="Unique name for wsrep_node_name")
    server_id: Optional[int] = Field(0, description="MySQL server-id (Auto-calculated if
0)")

    @field_validator('ip')
    @classmethod
    def validate_ip_format(cls, v: str):
        if not re.match(IP_REGEX, v):
            raise ValueError(f"Invalid IP format: '{v}'. Must be a valid IPv4 address.")
        return v #retursns the ip itself
class ClusterDeploymentRequest(BaseModel):
    # --- Infrastructure Groups ---
    mariadb_version: str = Field(..., pattern="^(10.6.16|10.6.21|10.11.16)$")
```

```
    galera_nodes: List[GaleraNodeConfig] = Field(..., min_length=3, max_length=3)
    lvs_ips: List[str] = Field(..., min_length=2, max_length=2)
    monitor_ip: str
    async_ip: str
    lvs_vip: str = Field(..., description="The Virtual IP that will float between LVS
nodes")

    repl_user: str = Field(default="repl_user")
    repl_password: str = Field(..., min_length=8)
    db_admin_password: str = Field(
        ...,
        min_length=8,
        description="Password for the MariaDB 'root' user. Must be unique from SST
password."
    )
    # --- Core Galera Parameters ---
    wsrep_cluster_name: str
    wsrep_on: str = "ON"
    wsrep_provider: str = "/usr/lib/galera/libgalera_smm.so"
    binlog_format: str = "ROW"
    default_storage_engine: str = "InnoDB"
    innodb_autoinc_lock_mode: int = 2
    bind_address: str = "0.0.0.0"
    # --- SST Settings ---
    wsrep_sst_method: str = "mariabackup"
    wsrep_sst_auth: str = Field(..., description="Credentials in 'user:password' format")
    # --- Version Specific Parameters ---
    wsrep_mode: Optional[str] = Field(default="REQUIRED_PRIMARY_KEY,STRICT_REPLICATION")
    binlog_expire_logs_seconds: Optional[int] = Field(default=604800)
    innodb_buffer_pool_instances: Optional[int] = Field(default=1)
    wsrep_slave_threads: Optional[int] = Field(default=4)
    innodb_undo_tablespaces: Optional[int] = Field(default=3)
    wsrep_gtid_domain_id: Optional[int] = Field(default=1)
    # Validate individual IP formats for standalone fields and lists
    @field_validator('monitor_ip', 'async_ip', 'lvs_vip', 'lvs_ips')
    @classmethod
    def validate_ips(cls, v):
        if isinstance(v, list):
            for ip in v:
                if not re.match(IP_REGEX, ip):
                    raise ValueError(f"Invalid IP format in list: '{ip}'")
        else:
            if not re.match(IP_REGEX, v):
                raise ValueError(f"Invalid IP format: '{v}'")
        return v
    @field_validator('galera_nodes')
    @classmethod
    def validate_unique_nodes(cls, v: List[GaleraNodeConfig]):
        names = [n.node_name for n in v]
        if len(set(names)) != len(names):
```

```
            raise ValueError("All wsrep_node_names must be unique.")
        return v
    @field_validator('wsrep_sst_auth')
    @classmethod
    def validate_sst_auth_format(cls, v: str):
        if ":" not in v:
            raise ValueError("SST Auth must be in 'user:password' format.")
        return v
    @model_validator(mode='after')
    def validate_password_uniqueness(self) -> 'ClusterDeploymentRequest':
        sst_password = self.wsrep_sst_auth.split(':')[1]
        if self.db_admin_password == sst_password:
            raise ValueError("Security Risk: Database Admin Password must be different
from SST Password.")
        return self
    @model_validator(mode='after')
    def validate_all_ips_unique(self) -> 'ClusterDeploymentRequest':
        """The 'Master Check': Ensures all 8 infrastructure IPs are unique."""
        all_ips = []

        # 1. Collect from Galera nodes (3 IPs)
        all_ips.extend([node.ip for node in self.galera_nodes])

        # 2. Collect from LVS nodes (2 IPs)
        all_ips.extend(self.lvs_ips)

        # 3. Collect standalone (3 IPs: Monitor, Async, VIP)
        all_ips.append(self.monitor_ip)
        all_ips.append(self.async_ip)
        all_ips.append(self.lvs_vip)
        # 4. Check for duplicates
        if len(all_ips) != len(set(all_ips)):
            seen = set()
            dupes = [x for x in all_ips if x in seen or seen.add(x)]
            raise ValueError(f"IP Collision Detected! The following IPs are reused: {',
'.join(set(dupes))}. "
                             "Each of the 8 infrastructure components must have a unique
IP.")

        return self
```

Main Python file

(venv) ubuntu@head-node:~/harbor-master$ sudo vim api/main.py

Summary of the script -
sys, os - to interact with os and modify system path
asyncio - core library for handling asynchronous code allowing servert to handling multiple task with out failing
Pymsql - for python to talk to mysql and mariadb
`pathlib (Path)`: A modern way to handle file system paths,
`RequestValidationError`: A specific error handler that catches issues when a user sends data that doesn't match your `ClusterDeploymentRequest` rules.
Json response to send back data
Streaming response to send back live logs
Corsmiddleware is a feature that allows browser to let frontend and backend interact
Header - for role based access
Run_in_threadpool - allows to run blocking heavy tasks in separate threads
Main_loop = a global variable used to store the brain of async systems o that background threads can send msg
Log_queue - queue to store logs till browser picks them up

Manages -
Schema validation
Calculates server id
Manages SSE and cors middleware
Validates the config
Preview and displays the config
Displays proper error messages
Prepares yml and inventory
Deploy the setup
Does health checks
Provision db and user has Principle of Least Privilege

```
None
import sys
import os
import json
import asyncio
import re
import pymysql
import logging
from pathlib import Path
from fastapi import FastAPI, Request, HTTPException
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse, StreamingResponse
from jinja2 import Template
from fastapi.middleware.cors import CORSMiddleware
from fastapi.concurrency import run_in_threadpool
```

```
from fastapi import Header
from typing import Optional
# --- Path Configuration ---
API_DIR = Path(__file__).resolve().parent
BACKEND_DIR = API_DIR.parent
ROOT_DIR = BACKEND_DIR.parent
sys.path.append(str(BACKEND_DIR))
# --- Internal Imports ---
from api.schemas import ClusterDeploymentRequest
from api.health_checker import HarborHealthChecker
from core.validator import verify_infrastructure
from core.generator import generate_ansible_files
from core.orchestrator import run_deployment
# --- Global State for SSE & Threading ---
main_loop = None
log_queue = asyncio.Queue()
def websocket_log_handler(message: str):
    """
    Callback used by Ansible. Cleans terminal codes and pushes
    the message into the async queue from the Ansible thread.
    """
    global main_loop
    if main_loop and main_loop.is_running():
        # Clean ANSI color codes
        clean_msg = re.sub(r'\x1b\[[0-9;]*m', '', message) # cleans the logs
        # Thread-safe push into the async log_queue
        main_loop.call_soon_threadsafe(log_queue.put_nowait, clean_msg)
# --- API Configuration ---
app = FastAPI(
    title="Harbor Master Control Plane",
    description="Enterprise MariaDB Galera & LVS Orchestrator (SSE Stream Mode)",
    version="2.0.0"
)
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
    handlers=[
        logging.StreamHandler(sys.stdout) # This is the "SRE Standard"
    ]
)
logger = logging.getLogger("HarborBackend")
@app.on_event("startup")
async def startup_event():
    global main_loop
    main_loop = asyncio.get_running_loop()
    print(">>> Harbor Master: Event Loop Captured for SSE")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],    #accept request from any website or ip
    allow_credentials=False,  #cannot share private data
```

```
    allow_methods=["*"],    #GET,POST,PUT,DELETE all allowed
    allow_headers=["*"],  #all types of extrs info can be sent
)
# --- SSE Endpoint ---
@app.get("/api/logs/stream")
async def stream_logs(request: Request):
    async def event_generator():
        while True:
            if await request.is_disconnected():  #closes the browser tab disconnected
                break
            try:
                log_chunk = await asyncio.wait_for(log_queue.get(), timeout=1.0) #waits
every 1 sec for new log
                lines = log_chunk.splitlines()
                for line in lines:
                    formatted_line = line.strip()
                    if formatted_line:
                        yield f"data: {formatted_line}\n\n"
            except asyncio.TimeoutError:    #constanly sending it keep connection on
                yield ": keep-alive\n\n"

    return StreamingResponse(event_generator(), media_type="text/event-stream")
def calculate_deterministic_server_id(ip_str: str) -> int:
    try:
        octets = ip_str.split('.')
        return (int(octets[2]) * 256) + int(octets[3])
    except (IndexError, ValueError):
        return hash(ip_str) % 65535  //check if math fails it creates ip via string
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    errors = []
    for error in exc.errors():
        errors.append({
            "loc": error["loc"],
            "msg": error["msg"],
            "type": error["type"]
        })
    return JSONResponse(status_code=400, content={"detail": errors})
@app.post("/validate")
async def validate_only(
    request: ClusterDeploymentRequest,
    x_user_role: Optional[str] = Header(None) # Captures 'role' from headers
):


    # 1. Prepare Server IDs
    for node in request.galera_nodes:
        node.server_id = calculate_deterministic_server_id(node.ip)
    # 2. Extract role (Default to 'viewer' if header is missing for safety)
    role = x_user_role.lower() if x_user_role else "viewer"
```

```
    # 3. Call the updated validator with the role
    success, val_message = verify_infrastructure(request, role)
    # 4. Handle Validation/Authorization Failures
    if not success:
        # Check if it's an Auth error or an Existence error to set status codes
        if "AUTHORIZATION_ERROR" in val_message:
            raise HTTPException(status_code=403, detail=val_message)
        raise HTTPException(status_code=400, detail=val_message)
    return {
        "status": "Validated",
        "message": val_message,
        "role_processed": role
    }
@app.post("/preview-config")
async def preview_config(request: ClusterDeploymentRequest):

    for node in request.galera_nodes:
        node.server_id = calculate_deterministic_server_id(str(node.ip))

    template_path = BACKEND_DIR / "ansible" / "roles" / "galera" / "templates" /
"galera.cnf.j2"
    if not template_path.exists():
        raise HTTPException(status_code=404, detail="Template not found")

    try:
        with open(template_path, 'r') as f:
            template_content = f.read()

        ips = [str(node.ip) for node in request.galera_nodes]
        cluster_address = f"gcomm://{','.join(ips)}"

        # Use the first node from the request as the context for the preview
        target_node = request.galera_nodes[0]

        template = Template(template_content)
        rendered_conf = template.render(
            **request.model_dump(mode='json'),
            wsrep_cluster_address=cluster_address,
            inventory_hostname=target_node.node_name,
            ansible_host=str(target_node.ip)
        )
        return {"filename": "60-galera.cnf", "content": rendered_conf}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
@app.post("/deploy")
async def trigger_deployment(request: ClusterDeploymentRequest,x_user_role: Optional[str]
= Header(None)):
    # Clear logs for a clean run
    while not log_queue.empty():
        log_queue.get_nowait()
```

```
    for node in request.galera_nodes:
        node.server_id = calculate_deterministic_server_id(str(node.ip))

    role = x_user_role.lower().strip() if x_user_role else "viewer"
    success, val_message = verify_infrastructure(request,role)
    if not success:
        # Check for Auth error here too as a safety net
        if "AUTHORIZATION_ERROR" in val_message:
            raise HTTPException(status_code=403, detail=val_message)
        raise HTTPException(status_code=400, detail=val_message)

    try:
        generate_ansible_files(request)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Generation Error: {str(e)}")

    try:
        websocket_log_handler("\n>>> System: Orchestration initializing via SSE...\n")
        status, return_code = await run_in_threadpool(
            run_deployment,
            log_callback=websocket_log_handler
        )
        if return_code != 0:
            websocket_log_handler(f"\n Ansible Error Detected: {status}\n")
            raise HTTPException(status_code=500, detail=f"Ansible Failed: {status}")

        websocket_log_handler("\n>>> System: Deployment Successful. Running
Diagnostics...\n")

        checker = HarborHealthChecker(request)
        health_report = checker.run_diagnostics()

        return {
            "status": "Success",
            "cluster_name": request.wsrep_cluster_name,
            "lvs_vip": str(request.lvs_vip),
            "health_report": health_report,
            "message": "Deployment complete."
        }

    except Exception as e:
        websocket_log_handler(f"\n System Error: {str(e)}\n")
        raise HTTPException(status_code=500, detail=f"System Error: {str(e)}")
@app.post("/api/create-db")
async def create_custom_db(data: dict):
    # 1. Normalize and Extract
    db_name = data.get('db_name', '').lower().strip()
    db_user = data.get('db_user', '').lower().strip()
    db_pass = data.get('db_password')
```

```
    role = data.get('role', 'Read-Only')
    ttl = data.get('ttl', 99)
    target_vip = data.get('vip')
    admin_pass = data.get('admin_password')
    try:
        ttl = int(ttl)
        if ttl < 1 or ttl > 365:
            # SRE Guardrail: Prevent absurdly short or year-plus TTLs
            ttl = 30 # Fallback to default
    except (ValueError, TypeError):
        ttl = 30 # Default safety
    # 2. SRE Guardrail: Forbidden Names Check
    FORBIDDEN = ['mysql', 'root', 'admin', 'sys', 'information_schema',
'performance_schema']
    if db_name in FORBIDDEN or db_user in FORBIDDEN:
        raise HTTPException(status_code=403, detail="Use of system-reserved names is
forbidden.")
    # 3. Map Roles
    role_map = {
        "Read-Only": "SELECT, SHOW VIEW",
        "Read-Write": "SELECT, INSERT, UPDATE, DELETE",
        "DDL-Admin": "ALL PRIVILEGES"
    }
    privs = role_map.get(role, "SELECT")
    try:
        conn = pymysql.connect(
            host=target_vip,
            user='root',
            password=admin_pass,
            autocommit=True
        )

        with conn.cursor() as cursor:
            # 4. Conflict Detection: User Check (STILL REQUIRED)
            # We never want to overwrite an existing user's password or grants
accidentally
            cursor.execute("SELECT User FROM mysql.user WHERE User = %s", (db_user,))
            if cursor.fetchone():
                raise HTTPException(status_code=400, detail="User already exists. Choose a
different username.")
            # 5. Database Presence Check (SOFT CHECK)
            cursor.execute("SELECT SCHEMA_NAME FROM information_schema.SCHEMATA WHERE
SCHEMA_NAME = %s", (db_name,))
            db_exists = cursor.fetchone()
            # 6. Execution Logic
            if not db_exists:
                # Scenario A: Brand new database
                cursor.execute(f"CREATE DATABASE `{db_name}`;")
                status_msg = f"Successfully created database {db_name} and user
{db_user}."
```

```
            else:
                # Scenario B: Existing database, new tenant
                status_msg = f"User {db_user} successfully attached to existing database
{db_name}."
            # 7. Create User and Grant (Works for both scenarios)
            grant_sql = f"GRANT {privs} ON `{db_name}`.* TO %s@'%%' IDENTIFIED BY %s"
            cursor.execute(grant_sql, (db_user, db_pass))
            # 8. Apply TTL
            expire_sql = f"ALTER USER %s@'%%' PASSWORD EXPIRE INTERVAL %s DAY"
            cursor.execute(expire_sql, (db_user, ttl))

            cursor.execute("FLUSH PRIVILEGES;")
        return {"status": "success", "detail": status_msg}
    except pymysql.Error as e:
        error_msg = e.args[1] if len(e.args) > 1 else str(e)
        raise HTTPException(status_code=500, detail=f"Database Engine Error: {error_msg}")
    except Exception as e:
        if isinstance(e, HTTPException): raise e
        raise HTTPException(status_code=500, detail=f"System Error: {str(e)}")
    finally:
        if 'conn' in locals(): conn.close()
```

Health Check File

(venv) **ubuntu@head-node-test**:**~/harbor-master/backend/api**$ sudo cat health_checker.py

- Ssh to all ips
- wsrep_cluster_size
- Wsrep_local_state_comment
- SHOW SLAVE STATUS;
- Slave_IO_Running
- Slave_SQL_Running
- ip addr show | grep {vip} - checks whose holding the vip
- sudo ipvsadm -Ln -

None

```
import pymysql
import paramiko
import logging
import re
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("HarborHealth")
```

```
class HarborHealthChecker:
    def __init__(self, request):
        self.request = request
        self.results = {
            "galera": [],
            "async": None,
            "lvs": [],
            "active_writer_ip": None,
            "cluster_size": 0,
            "overall_status": "Healthy",
            "debug_logs": []
        }
    def _log_debug(self, msg):
        logger.info(msg)
        self.results["debug_logs"].append(msg)
    def _check_mariadb(self, host, user, password, is_async=False):
        label = "Async" if is_async else "Galera"
        self._log_debug(f"--- Checking {label} Node: {host} ---")

        try:
            conn = pymysql.connect(
                host=str(host), user=user, password=password,
                connect_timeout=5, cursorclass=pymysql.cursors.DictCursor
            )
            self._log_debug(f"  [PASS] MySQL Connection established.")

            with conn.cursor() as cursor:
                if not is_async:
                    # Check 1: Cluster Size
                    cursor.execute("SHOW STATUS LIKE 'wsrep_cluster_size';")
                    size = cursor.fetchone()
                    c_size = int(size['Value']) if size else 0
                    self.results["cluster_size"] = c_size
                    self._log_debug(f"  [DATA] wsrep_cluster_size: {c_size}")
                    # Check 2: Sync State
                    cursor.execute("SHOW STATUS LIKE 'wsrep_local_state_comment';")
                    state = cursor.fetchone()
                    s_val = state['Value'] if state else "Unknown"
                    self._log_debug(f"  [DATA] wsrep_local_state_comment: {s_val}")

                    return {"host": str(host), "reachable": True, "sync_state": s_val}
                else:
                    # Check 3: Async Slave Status
                    cursor.execute("SHOW SLAVE STATUS;")
                    replica = cursor.fetchone()
                    if replica:
                        io = replica['Slave_IO_Running']
                        sql = replica['Slave_SQL_Running']
                        self._log_debug(f"  [DATA] Slave_IO_Running: {io}")
                        self._log_debug(f"  [DATA] Slave_SQL_Running: {sql}")
```

```
                        return {"host": str(host), "reachable": True, "io": io, "sql":
sql}
                    self._log_debug("  [FAIL] No replication status found.")
                    return {"host": str(host), "reachable": True, "io": "No", "sql": "No"}
        except Exception as e:
            self._log_debug(f"  [FAIL] Connection Error: {str(e)}")
            self.results["overall_status"] = "Degraded"
            return {"host": str(host), "reachable": False}
    def _check_lvs_ssh(self, lvs_ip, vip):
        self._log_debug(f"--- Checking LVS Node: {lvs_ip} ---")
        try:
            ssh = paramiko.SSHClient()
            ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
            ssh.connect(str(lvs_ip), username='ubuntu',
key_filename="/home/ubuntu/.ssh/id_ansible", timeout=5)
            self._log_debug(f"  [PASS] SSH Connection established.")
            # Check 1: VIP Ownership
            _, stdout, _ = ssh.exec_command(f"ip addr show | grep {vip}")
            has_vip = bool(stdout.read().decode().strip())
            self._log_debug(f"  [DATA] Holds VIP ({vip}): {has_vip}")
            # Check 2: IPVS Table Audit
            _, stdout, _ = ssh.exec_command("sudo ipvsadm -Ln")
            ipvs_output = stdout.read().decode()
            self._log_debug(f"  [STEP] Parsing `ipvsadm -Ln` output...")
            active_ip = None
            lines = ipvs_output.split('\n')
            for i, line in enumerate(lines):
                if vip in line:
                    for sub_line in lines[i+1:]:
                        if "->" in sub_line:
                            parts = sub_line.split()
                            server_ip = parts[1].split(':')[0]
                            weight = int(parts[3])
                            if weight > 0:
                                active_ip = server_ip
                                self._log_debug(f"  [DATA] Found Active Backend:
{active_ip} (Weight: {weight})")
                                break

            if not active_ip:
                self._log_debug(f"  [WARN] No active backends found in IPVS for VIP
{vip}")
            ssh.close()
            return {"host": str(lvs_ip), "ssh": True, "vip": has_vip, "target": active_ip}
        except Exception as e:
            self._log_debug(f"  [FAIL] SSH Error: {str(e)}")
            return {"host": str(lvs_ip), "ssh": False}
    def run_diagnostics(self):
        self._log_debug("Starting Harbor Master Diagnostic Suite...")
```

```
        mysql_pass = self.request.db_admin_password if hasattr(self.request,
'db_admin_password') else ""
        # Tier 1: Galera
        for node in self.request.galera_nodes:
            self.results["galera"].append(self._check_mariadb(node.ip, "root",
mysql_pass))
        # Tier 2: LVS
        lvs_targets = {}
        for lvs_ip in self.request.lvs_ips:
            res = self._check_lvs_ssh(lvs_ip, self.request.lvs_vip)
            self.results["lvs"].append(res)
            if res.get("target"):
                lvs_targets[res["host"]] = res["target"]
        # Tier 3: Async
        if self.request.async_ip:
            self.results["async"] = self._check_mariadb(self.request.async_ip, "root",
mysql_pass, is_async=True)
        # Cross-Tier Consensus Logic
        unique_targets = set(lvs_targets.values())
        if len(unique_targets) == 1:
            self.results["active_writer_ip"] = list(unique_targets)[0]
            self._log_debug(f"CONSENSUS: All LVS nodes routing to
{self.results['active_writer_ip']}")
        elif len(unique_targets) > 1:
            self.results["overall_status"] = "CRITICAL"
            self._log_debug(f"CRITICAL: Split-brain detected! LVS targets: {lvs_targets}")
        self._log_debug(f"Final Cluster Status: {self.results['overall_status']}")
        return self.results
```

Generator File

(venv) ubuntu@head-node:~/harbor-master$ sudo vim core/generator.py

Take structured data from the api and converts it into physical yaml files
Organizes your servers into logical groups (Galera, LVS, Monitor) so Ansible knows which tasks to run on which machine.
Creates inventory - host.yml and extravars.yml

```
None
import os
```

```
import yaml
from pathlib import Path
def generate_ansible_files(request_data):


    # --- 1. SETUP PATHS ---
    # Current file: backend/core/generator.py
    # BASE_DIR should be 'backend'
    current_file = Path(__file__).resolve()
    base_dir = current_file.parent.parent  # Points to /backend

    inventory_dir = base_dir / 'ansible' / 'inventory'
    os.makedirs(inventory_dir, exist_ok=True)
    # Helper function to match the server_id logic in main.py
    def get_id(ip_str):
        try:
            octets = str(ip_str).split('.')
            return (int(octets[2]) * 256) + int(octets[3])
        except (IndexError, ValueError):
            return hash(str(ip_str)) % 4294967295
    # --- 2. BUILD THE INVENTORY (hosts.yml) ---
    inventory = {
        'all': {
            'children': {
                'galera': {
                    'hosts': {
                        node.node_name: {
                            'ansible_host': str(node.ip),
                            'server_id': node.server_id  # Passed from main.py
                        } for node in request_data.galera_nodes
                    }
                },
                'lvs': {
                    'hosts': {
                        f"lvs-node-{i+1}": {'ansible_host': str(ip)}
                        for i, ip in enumerate(request_data.lvs_ips)
                    }
                },
                'async': {
                    'hosts': {
                        'async-node': {
                            'ansible_host': str(request_data.async_ip),
                            'server_id': get_id(request_data.async_ip)
                        }
                    }
                },
                'monitoring': {
                    'hosts': {
                        'monitor-node': {'ansible_host': str(request_data.monitor_ip)}
                    }
```

```
            }
        },
        'vars': {
            'ansible_user': 'ubuntu',
            # Updated to use the standard Ubuntu home path
            'ansible_ssh_private_key_file': '/home/ubuntu/.ssh/id_ansible',
            'ansible_ssh_common_args': '-o StrictHostKeyChecking=no'
        }
    }
}
# --- 3. BUILD THE VARIABLES (extravars.yml) ---
# Convert Pydantic model to a standard dictionary
extravars = request_data.model_dump(mode='json')
extravars['hcip_value'] = "10.0.0.100"
# Calculate derived values needed for Galera templates
ips = [str(node.ip) for node in request_data.galera_nodes]
extravars['wsrep_cluster_address'] = f"gcomm://{','.join(ips)}"

# Simplify the node list for easy iteration in Ansible Jinja2 templates
extravars['galera_node_list'] = [
    {"name": node.node_name, "ip": str(node.ip), "id": node.server_id}
    for node in request_data.galera_nodes
]
# --- 4. WRITE TO DISK ---
try:
    # Write inventory file
    inventory_file = inventory_dir / 'hosts.yml'
    with open(inventory_file, 'w') as f:
        yaml.dump(inventory, f, default_flow_style=False)

    # Write extravars file
    vars_file = inventory_dir / 'extravars.yml'
    with open(vars_file, 'w') as f:
        yaml.dump(extravars, f, default_flow_style=False)

    print(f"DEBUG: Ansible files generated successfully in {inventory_dir}")
    return True
except Exception as e:
    print(f"ERROR: Failed to write Ansible files: {str(e)}")
    raise e
```

Validator File

(venv) ubuntu@head-node:~/harbor-master$ sudo vim core/validator.py

Deep Resource Discovery
OS & Version Compatibility
Conflict & Existence Detection:
Security Enforcement:
Path Intelligence:
Role-Based Access Control (RBAC):
Configuration Sanity Checks
Connectivity Verification:

None

```
import re
import subprocess
import paramiko
import os
import socket
from typing import List, Dict, Tuple
class InfrastructureValidator:
    def __init__(self):
        # Path to the SSH key on the Head Node
        self.key_path = "/home/ubuntu/.ssh/id_ansible"

        # Regex: 8+ chars, 1 Upper, 1 Lower, 1 Number, 1 Special
        self.pass_regex =
r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$"
    def check_ping(self, ip) -> bool:
        """Standard ICMP ping check."""
        try:
            res = subprocess.run(['ping', '-c', '1', '-W', '1', str(ip)],
capture_output=True)
            return res.returncode == 0
        except Exception:
            return False
    def is_port_open(self, ip, port=3306) -> bool:
        """Checks if MariaDB port is already occupied."""
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.settimeout(1)
                return s.connect_ex((str(ip), port)) == 0
        except Exception:
            return False

       def _get_cluster_health(self, ip: str, password: str) -> Dict:
        """Queries an existing MariaDB instance ONLY for Galera health metrics."""
        health = {"status": "Disconnected", "size": "0", "ready": "OFF"}
        ssh = paramiko.SSHClient()
        ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        try:
            ssh.connect(str(ip), username="ubuntu", key_filename=self.key_path, timeout=3)
            # Strictly Galera status checks
            cmd = (
```

```
                f"mariadb -u root -p'{password}' -e \"SHOW STATUS LIKE
'wsrep_local_state_comment'; "
                "SHOW STATUS LIKE 'wsrep_cluster_size'; "
                "SHOW STATUS LIKE 'wsrep_ready';\""
            )
            stdin, stdout, stderr = ssh.exec_command(cmd)
            output = stdout.read().decode()
            if "wsrep_local_state_comment" in output:
                for line in output.splitlines():
                    if "wsrep_local_state_comment" in line: health["status"] =
line.split()[-1]
                    if "wsrep_cluster_size" in line: health["size"] = line.split()[-1]
                    if "wsrep_ready" in line: health["ready"] = line.split()[-1]
            ssh.close()
        except Exception:
            health["status"] = "Unknown"
        return health
    def get_remote_resources(self, ip) -> Dict:
        """
        Connects via SSH to fetch actual CPU, RAM, and Disk from the target VM.
        This ensures the VM isn't under-provisioned before Ansible starts.
        """
        resources = {
            "os_family": "unknown",
            "os_version": "unknown",
            "cpus": 0,
            "ram_gb": 0.0,
            "disk_gb": 0.0
        }
        ssh = paramiko.SSHClient()
        ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

        try:
            ssh.connect(str(ip), username="ubuntu", key_filename=self.key_path, timeout=5)

            # 1. Check OS Family
            stdin, stdout, stderr = ssh.exec_command("cat /etc/os-release")
            os_out = stdout.read().decode().lower()
            if any(x in os_out for x in ["debian", "ubuntu"]):
                resources["os_family"] = "debian"
                stdin, stdout, stderr = ssh.exec_command("lsb_release -rs")
                resources["os_version"] = stdout.read().decode().strip()
            elif any(x in os_out for x in ["rhel", "centos", "alma", "rocky"]):
                resources["os_family"] = "redhat"
            # 2. Check CPU Cores
            stdin, stdout, stderr = ssh.exec_command("nproc")
            resources["cpus"] = int(stdout.read().decode().strip())
            # 3. Check RAM (Total GB)
            stdin, stdout, stderr = ssh.exec_command("grep MemTotal /proc/meminfo | awk
'{print $2}'")
```

```
            mem_kb = int(stdout.read().decode().strip())
            resources["ram_gb"] = round(mem_kb / (1024 * 1024), 2)
            # 4. Check Disk (Root partition GB)
            stdin, stdout, stderr = ssh.exec_command("df / --output=size | tail -1")
            disk_kb = int(stdout.read().decode().strip())
            resources["disk_gb"] = round(disk_kb / (1024 * 1024), 2)
            ssh.close()
        except Exception as e:
            print(f"SSH Resource Discovery Error on {ip}: {e}")

        return resources
    def validate_all(self, request, role: str) -> Tuple[bool, str]:
        """Main validation logic matching ClusterDeploymentRequest schema."""
        # 1. Connectivity Checks
        galera_ips = [str(n.ip) for n in request.galera_nodes]
        lvs_ips = [str(ip) for ip in request.lvs_ips]
        all_ips = galera_ips + lvs_ips + [str(request.monitor_ip), str(request.async_ip)]

        for ip in all_ips:
            if not self.check_ping(ip):
                return False, f"Connectivity Error: Node {ip} is unreachable (Ping
failed)."
        # 2. Hardware Resource & OS Path Check
        # We check the first Galera node as a representative for the cluster hardware
        # --- SAFE LIMIT VALIDATION ---
        # These match the limits we set to keep your laptop from crashing
       # if remote["cpus"] < 1:
        #    return False, f"Resource Error: {galera_ips[0]} has only {remote['cpus']}
CPU. Minimum 1 required."

       # if remote["ram_gb"] < 1.7: # Allowing a small buffer for 2GB VMs
        #    return False, f"Resource Error: {galera_ips[0]} has only {remote['ram_gb']}GB
RAM. Minimum 2GB required."

       # if remote["disk_gb"] < 8.0:
        #    return False, f"Resource Error: {galera_ips[0]} has only
{remote['disk_gb']}GB Disk. Minimum 20GB required."
        # OS Path Check for wsrep_provider
        deployment_nodes = galera_ips

        occupied_nodes = [ip for ip in deployment_nodes if self.is_port_open(ip)]
        if occupied_nodes:

            health_reports = []
            for ip in occupied_nodes:
                h = self._get_cluster_health(ip, request.db_admin_password)
                node_report = f"{ip} (Galera): {h['status']}, Size: {h['size']}"
                health_reports.append(f"[{node_report}]")
            report_str = " ".join(health_reports)
```

```
            # This will now block Admins as well if MariaDB is already running
            return False, f"EXISTS|Conflict Error: MariaDB active. {report_str}."
        # Authorization check for viewers happens AFTER the conflict check
        if role == "viewer":
            return False, "AUTHORIZATION_ERROR|You are not authorized to create a new
cluster."

        remote = self.get_remote_resources(galera_ips[0])
        if remote["os_family"] == "unknown":
            return False, f"SSH Error: Could not authenticate with {galera_ips[0]}. Check
your SSH keys."
        db_version = request.mariadb_version # e.g., "10.6.16"
        os_version = remote["os_version"]    # e.g., "24.04"
        # Rule: 10.6 is not reliable on 24.04 due to OpenSSL 3.0 conflicts
        if db_version.startswith("10.6") and os_version == "24.04":
            return False, (f"Compatibility Error: MariaDB 10.6 is not supported on Ubuntu
24.04. "
                           f"Node {galera_ips[0]} is running 24.04. Please select MariaDB
10.11.")
        # Rule: 10.11 is optimized for 22.04 and 24.04
        if db_version.startswith("10.11") and os_version == "20.04":
            return False, (f"Compatibility Error: MariaDB 10.11 requires Ubuntu 22.04+. "
                           f"Node {galera_ips[0]} is running an older OS ({os_version}).")
        if remote["os_family"] == "debian":
            expected_path = "/usr/lib/galera/libgalera_smm.so"
        elif remote["os_family"] == "redhat":
            expected_path = "/usr/lib64/galera/libgalera_smm.so"
        else:
            expected_path = request.wsrep_provider

        if request.wsrep_provider != expected_path:
            return False, f"Path Error: For {remote['os_family']}, wsrep_provider should
be {expected_path}"
        # 3. Password Complexity (SST Auth)
        try:
            sst_pass = request.wsrep_sst_auth.split(":")[1]
            if not re.match(self.pass_regex, sst_pass):
                return False, "Security Error: SST Password does not meet complexity
standards (Upper, Lower, Num, Special)."
            if not re.match(self.pass_regex, request.db_admin_password):
                return False, "Security Error: Database Admin Password does not meet
complexity standards."
            # Ensure they are not the same
            if sst_pass == request.db_admin_password:
                return False, "Security Error: Database Admin Password and SST Password
must be different."
        except IndexError:
            return False, "Format Error: wsrep_sst_auth must be 'user:password'."
        # 4. Global MariaDB Requirements
        if request.binlog_format.upper() != "ROW":
```

```
            return False, "Config Error: binlog_format must be 'ROW'."
        if request.default_storage_engine.lower() != "innodb":
            return False, "Config Error: default_storage_engine must be 'InnoDB'."
        if int(request.innodb_autoinc_lock_mode) != 2:
            return False, "Config Error: innodb_autoinc_lock_mode must be 2."
        # 5. Version-Specific Logic
        v = request.mariadb_version
        if not request.binlog_expire_logs_seconds or request.binlog_expire_logs_seconds <
3600:
            return False, "Config Error: binlog_expire_logs_seconds must be at least 3600
(1 hour)."
        if "REQUIRED_PRIMARY_KEY" not in (request.wsrep_mode or ""):
            return False, "Security Error: 10.6+ requires
wsrep_mode='REQUIRED_PRIMARY_KEY' for stability."
        # 6. Final Server ID Integrity Check
        generated_ids = [n.server_id for n in request.galera_nodes]
        if len(generated_ids) != len(set(generated_ids)):
            return False, "Conflict: Auto-generated Server IDs must be unique across the
cluster."

        return True, "All checks passed. Ready for deployment."
def verify_infrastructure(request, role: str):
    validator = InfrastructureValidator()
    return validator.validate_all(request, role)
```

Orchestrator File

(venv) ubuntu@head-node:~/harbor-master$ sudo vim core/orchestrator.py

Gather the inventory, variables, and playbooks into one execution command.
Uses `ansible-runner` to manage the background process of SSH-ing into remote servers.
Monitors every single task Ansible performs and "pipes" that information to your UI via the `event_handler`.
Captures the success or failure code so your API can tell the user if the cluster is ready or if it needs troubleshooting.

```
None
import ansible_runner
import os
import yaml
from pathlib import Path
def run_deployment(log_callback=None):
```

```
    """
    Executes the Ansible playbook using ansible-runner.
    Coordinates the inventory and extravars generated in the previous step.

    :param log_callback: A function to handle real-time log streaming (WebSocket bridge)
    """
    # --- 1. SETUP PATHS ---
    current_file = Path(__file__).resolve()
    base_dir = current_file.parent.parent  # Points to /backend
    playbook_path = base_dir / 'ansible' / 'deploy_cluster.yml'
    inventory_path = base_dir / 'ansible' / 'inventory' / 'hosts.yml'
    vars_path = base_dir / 'ansible' / 'inventory' / 'extravars.yml'
    # --- 2. LOAD EXTRA VARIABLES ---
    extravars = {}
    if vars_path.exists():
        try:
            with open(vars_path, 'r') as f:
                extravars = yaml.safe_load(f)
        except Exception as e:
            print(f"ERROR: Could not parse {vars_path}: {e}")
    else:
        print(f"WARNING: {vars_path} not found. Running without extra vars.")
    # --- 3. DEFINE EVENT HANDLER FOR REAL-TIME LOGS ---
    # --- Inside core/orchestrator.py ---
    def event_handler(event):
        if log_callback:
            # 'stdout' contains the actual formatted line (e.g., "TASK [monitoring :
...]")
            stdout = event.get('stdout')
            if stdout:
                # We strip extra trailing newlines to keep the UI tidy
                log_callback(stdout.rstrip())
        return True
    # --- 4. LAUNCH ANSIBLE RUNNER ---
    print(f"\nLaunching Ansible Runner...")
    print(f"Private Data Dir: {base_dir}")
    print(f"Playbook: {playbook_path}")
    # We use event_handler to capture live output for the UI
    r = ansible_runner.run(
        private_data_dir=str(base_dir),
        playbook=str(playbook_path),
        inventory=str(inventory_path),
        extravars=extravars,
        event_handler=event_handler, # The Real-Time Hook
        quiet=False,
    )
    # --- 5. RETURN RESULTS ---
    print(f"\n--- Deployment Summary ---")
    print(f"✅ Final Status: {r.status}")
    print(f"🔢 Return Code: {r.rc}")
```

```
    print(f"-------------------------\n")
    return r.status, r.rc
```

Deploy Play Book

(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/deploy_cluster.yml

```
None
---
# ================================================================
# PHASE 0: INFRASTRUCTURE HANDSHAKE
# ================================================================
- name: "PHASE: PRE_FLIGHT_START"
  hosts: localhost
  connection: local
  gather_facts: false
  tasks:
    - name: Ensure LVS files directory exists
      file:
        path: "{{ playbook_dir }}/roles/lvs/files"
        state: directory
        mode: '0755'     #owner can read/write/execute
    - name: Generate SSH keypair for LVS Health Checks
      openssh_keypair:      #module that generates public and private keys
        path: "{{ playbook_dir }}/roles/lvs/files/id_rsa"
        type: rsa     #encription alof and it s strength 4096
        size: 4096    #if key already exits it wont overwrite it
        force: false
      register: key_gen  #saves output to a variable name key_gen
# ================================================================
# PHASE 1: OS PREPARATION
# ================================================================
- name: "PHASE: MARIADB_PREP_START"
  hosts: all
  serial: 2
  become: yes
  tasks:
    - name: Wait for apt lock to be released
      shell: "while fuser /var/lib/dpkg/lock-frontend >/dev/null 2>&1; do sleep 5; done;"
      timeout: 300 #waits for automatic update running in the backgroud to end
```

```
    - name: Install basic utilities
      apt:
        name: [curl, wget, software-properties-common, gnupg2]
        state: present
        update_cache: yes
- name: Phase 2 - Setup MariaDB Repositories
  hosts: galera, async
  become: yes
  serial: 2
  roles:
    - common
# ================================================================
# PHASE 3: GALERA CLUSTER (The Core)
# ================================================================
- name: "PHASE: GALERA_SETUP_START"
  hosts: galera
  become: yes
  serial: 1
  roles:
    - galera
# ================================================================
# PHASE 4: HIGH AVAILABILITY
# ================================================================
- name: "PHASE: LVS_SETUP_START"
  hosts: lvs
  become: yes
  roles:
    - lvs
# ================================================================
# PHASE 5: ASYNC REPLICATION
# ================================================================
- name: "PHASE: ASYNC_SETUP_START"
  hosts: async
  become: yes
  roles:
    - async_replica
# ================================================================
# PHASE 6: MONITORING STACK
# ================================================================
- name: "PHASE: MONITORING_SETUP_START"
  hosts: all
  become: yes
  serial: 2
  roles:
    - monitoring
- name: "PHASE: DEPLOYMENT_COMPLETE"
  hosts: localhost
  connection: local
  tasks:
    - debug:
```

```
        msg: "Harbor Master Deployment Finished Successfully."
```

Common Maria db Setup

(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/common/tasks/main.yml

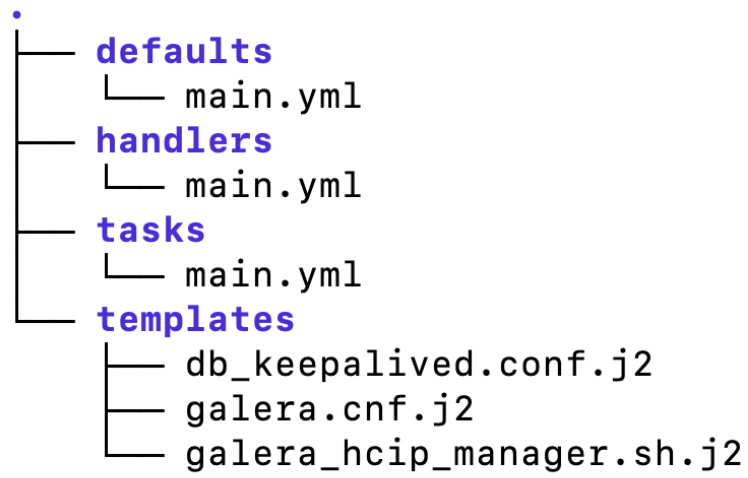
```
None
---
# ============================================================
# PHASE 0: VERSION VALIDATION & IDEMPOTENCY GUARD
# ============================================================
- name: Check if MariaDB is already installed
  command: mysql --version
  register: mariadb_check
  failed_when: false
  changed_when: false
- name: Extract installed version string
  set_fact:
    # UPDATED REGEX: Captures X.Y and optional .Z (e.g., 10.6 or 10.6.21)
    installed_version: "{{ mariadb_check.stdout | regex_search('Distrib
([0-9]+\\.[0-9]+(\\.[0-9]+)?)', '\\1') | first | default(mariadb_check.stdout |
regex_search('([0-9]+\\.[0-9]+(\\.[0-9]+)?)-MariaDB', '\\1') | first) }}"
  when: mariadb_check.rc == 0
- name: Block execution if versions mismatch
  fail:
    msg: "Safety Stop: You requested MariaDB {{ mariadb_version }}, but version {{
installed_version }} is already installed. Downgrades/Upgrades are not allowed."
  when:
    - mariadb_check.rc == 0
    - installed_version is defined
    - not (mariadb_version | string).startswith(installed_version)
  run_once: false
# ============================================================
# PHASE 1: REPOSITORY & DEPENDENCY SETUP
# ============================================================
- name: "PHASE: PRE_FLIGHT_START"
  debug:
    msg: "Starting Pre-flight configuration on Head Node"
- name: Update apt cache
  apt:
```

```
    update_cache: yes
    cache_valid_time: 3600 #dont run if ran one hr ago
  become: yes
# 1. PREPARE GPG KEYRINGS
- name: Ensure keyrings directory exists
  file:
    path: /etc/apt/keyrings
    state: directory
    mode: '0755'
  become: yes
- name: Add MariaDB GPG key
  get_url:
    url: "https://mariadb.org/mariadb_release_signing_key.asc"
    dest: /etc/apt/keyrings/mariadb.asc
    mode: '0644' #set permissions so only system can read it and root can change it
  become: yes
# 2. DYNAMIC LOGIC (The "Brain" of the playbook)
- name: Set MariaDB variables based on version
  set_fact:
    # 10.5.16 is the "special child" that needs Focal
    repo_dist: "{{ 'focal' if mariadb_version == '10.5.16' else
ansible_distribution_release }}"
    ver_suffix: "{{ 'focal' if mariadb_version == '10.5.16' else 'ubu2204' }}"

- name: Construct full version string
  set_fact:
    # This creates strings like 1:10.11.9+maria~ubu2204 or 1:10.5.16+maria~focal
    full_mariadb_ver: "1:{{ mariadb_version }}+maria~{{ ver_suffix }}"
# 3. ADD REPOSITORY
- name: Add MariaDB Repository
  apt_repository:
    repo: "deb [arch=arm64,amd64 signed-by=/etc/apt/keyrings/mariadb.asc]
https://archive.mariadb.org/mariadb-{{ mariadb_version }}/repo/ubuntu {{ repo_dist }}
main"
    state: present
    filename: mariadb_archive
  become: yes
  register: repo_added
# 4. PINNING
- name: Pin MariaDB Archive Repository
  copy:
    content: |
      Package: *
      Pin: origin archive.mariadb.org
      Pin-Priority: 1001
    dest: /etc/apt/preferences.d/mariadb
  become: yes
# 5. INSTALL
- name: Install MariaDB Packages
  apt:
```

```
    name:
      - "mariadb-server={{ full_mariadb_ver }}"  #mysqld
      - "mariadb-client={{ full_mariadb_ver }}" #cli tools
      - "libmariadb3={{ full_mariadb_ver }}" #c client library used by python to talkto db
      - "mariadb-common={{ full_mariadb_ver }}" #conatins base directory /etc/mysql
      - "mariadb-backup={{ full_mariadb_ver }}"
      - python3-pymysql   #python mysql client library
      - socat #for data transfer during sst
    state: present
    update_cache: yes
    allow_downgrade: yes
  become: yes
# 6. HOLD PACKAGES
- name: Hold MariaDB packages
  dpkg_selections:
    name: "{{ item }}"
    selection: hold
  loop:
    - mariadb-server
    - mariadb-client
    - libmariadb3
    - mariadb-common
    - mariadb-backup
  become: yes
- name: Update apt cache after adding repo
  apt:
    update_cache: yes
  become: yes
  when: repo_added.changed
```

Galera Setup



(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/galera/defaults/main.yml

```
None
---
hcip_value: "10.0.0.100"
```

(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/galera/tasks/main.yml

```
None
---
# ================================================================
# PHASE 0: DISCOVERY (Determine the Current Reality)
# ================================================================
- name: "DISCOVERY: Audit all nodes (Pre-Flight)"
  setup:
  delegate_to: "{{ item }}"
  delegate_facts: true
  loop: "{{ groups['galera'] }}" #run and gather facts from each galera node
  run_once: true
- name: "DISCOVERY: Check if MariaDB is installed"
  stat:
    path: /usr/sbin/mariadbd
  register: mariadb_installed
- name: "DISCOVERY: Check local Galera state"
```

```
  shell: |
    STATUS=$(mariadb -u root -N -s -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"
2>/dev/null | awk '{print $2}')
    if [ -z "$STATUS" ]; then
      PASS="{{ db_admin_password }}"
      STATUS=$(mariadb -u root -p"$PASS" -N -s -e "SHOW STATUS LIKE
'wsrep_local_state_comment';" 2>/dev/null | awk '{print $2}')
    fi
    echo "${STATUS:-OFFLINE}"
  become: yes
  register: galera_info
  changed_when: false
- name: "LOGIC: Set Discovery Facts"
  set_fact:
    is_installed: "{{ mariadb_installed.stat.exists }}"
    is_synced: "{{ 'Synced' in galera_info.stdout }}"
    # We check if ANY node in the cluster is already healthy
    cluster_is_alive: "{{ groups['galera'] | map('extract', hostvars) |
selectattr('galera_info.stdout', 'defined') | selectattr('galera_info.stdout', 'search',
'Synced') | list | length > 0 }}"
# ================================================================
# PHASE 1: PREPARATION (Install & Config)
# ================================================================
- name: "INSTALL: MariaDB Packages"
  apt:
    name: [mariadb-server, mariadb-client, mariadb-backup, python3-pymysql, socat, rsync,
keepalived]
    state: present
    update_cache: yes
  become: yes
  when: not is_installed
- name: "CONFIG: Deploy Galera Configuration"
  template:
    src: galera.cnf.j2
    dest: /etc/mysql/mariadb.conf.d/60-galera.cnf
    owner: root
    group: root
    mode: '0644'
  become: yes
  register: config_deployed
# ================================================================
# PHASE 2: CLUSTER INITIALIZATION
# ================================================================
- name: "STOP: Ensure MariaDB is stopped before cluster logic"
  service:
    name: mariadb
    state: stopped
  become: yes
  when: not is_synced
- name: "CASE A: Bootstrap Primary Node"
```

```
  block:
    - name: "CHECK: Check if grastate.dat exists"
      stat:
        path: /var/lib/mysql/grastate.dat
      register: grastate_file
    - name: "Force safe_to_bootstrap"
      lineinfile:
        path: /var/lib/mysql/grastate.dat
        regexp: '^safe_to_bootstrap: 0'
        line: 'safe_to_bootstrap: 1'
      when: grastate_file.stat.exists
    - name: "Initialize Galera New Cluster"
      command: galera_new_cluster
  become: yes
  # Only bootstrap if the cluster is totally dead AND I am node 1
  when:
    - not is_synced
    - not cluster_is_alive
    - inventory_hostname == groups['galera'][0]
- name: "CASE B: Join Existing Cluster"
  service:
    name: mariadb
    state: started
  become: yes
  # Run this if the cluster is alive OR I am not node 1
  when:
    - not is_synced
    - (cluster_is_alive or inventory_hostname != groups['galera'][0])
# ================================================================
# PHASE 3: NETWORKING & HA
# ================================================================
- name: "LVS-DR & HCIP Networking"
  block:
    - name: "LVS-DR: Loopback VIP & ARP Suppression"
      shell: |
        ip addr add {{ lvs_vip }}/32 dev lo || true
        # Tells the kernel not to announce the VIP to the network on any interface
        sysctl -w net.ipv4.conf.all.arp_ignore=1
        sysctl -w net.ipv4.conf.all.arp_announce=2

        # Specifically protect the loopback interface
        sysctl -w net.ipv4.conf.lo.arp_ignore=1
        sysctl -w net.ipv4.conf.lo.arp_announce=2
      changed_when: false

    - name: "HCIP: Interface dummy0 Setup"
      shell: |
        modprobe dummy && ip link add dummy0 type dummy || true
        ip link set dummy0 up
      changed_when: false
```

```
    - name: "HCIP: Install Keepalived"
      apt:
        name: keepalived
        state: present
        update_cache: yes

    - name: "HCIP: Ensure Keepalived directory exists"
      file:
        path: /etc/keepalived
        state: directory
        mode: '0755'
    - name: "HCIP: Deploy Keepalived Files"
      template:
        src: "{{ item.src }}"
        dest: "{{ item.dest }}"
        mode: "{{ item.mode }}"
      loop:
        - { src: 'galera_hcip_manager.sh.j2', dest:
'/usr/local/sbin/galera_healthcheck.sh', mode: '0755' }
        - { src: 'db_keepalived.conf.j2', dest: '/etc/keepalived/keepalived.conf', mode:
'0644' }
      register: ha_files
    - name: "HCIP: Ensure Keepalived service"
      service:
        name: keepalived
        # Using 'restarted' logic is good, but Ensure it's 'started' if it was just
installed
        state: "{{ 'restarted' if ha_files.changed else 'started' }}"
        enabled: yes
  become: yes
  when: inventory_hostname in groups['galera'][:2]
# ============================================================
# PHASE 4: FINAL VALIDATION & USERS
# ============================================================
- name: "WAIT: Ensure node is Synced before finishing"
  shell: |
    # CHANGED: Using db_admin_password
    PASS="{{ db_admin_password }}"
    mariadb -u root -p"$PASS" -N -s -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"
  become: yes
  register: final_sync
  until: '"Synced" in final_sync.stdout'
  retries: 30
  delay: 10
  when: not is_synced
- name: "SSH: Authorize LVS Health Check Key"
  ansible.builtin.authorized_key:
    user: root
    state: present
    # This pulls the public key from your head-node's LVS role folder
```

```
    key: "{{ lookup('file', 'roles/lvs/files/id_rsa.pub') }}"
  become: yes

- name: "USERS: Setup Cluster Access"
  community.mysql.mysql_user:
    # Logic: If item.name is root, use db_admin_password; otherwise use SST password
    name: "{{ item.name }}"
    password: "{{ (item.name == 'root') | ternary(db_admin_password,
wsrep_sst_auth.split(':')[1]) }}"
    host: "{{ item.host }}"
    # Logic: Give root ALL privileges, give SST user only sync privileges
    priv: "{{ (item.name == 'root') | ternary('*.*:ALL,GRANT', '*.*:RELOAD,PROCESS,LOCK
TABLES,REPLICATION CLIENT') }}"
    state: present
    login_user: root

    login_password: "{{ db_admin_password }}"
    login_unix_socket: /var/run/mysqld/mysqld.sock
    check_implicit_admin: yes
  loop:
    - { name: "{{ wsrep_sst_auth.split(':')[0] }}", host: "%" }
    - { name: "root", host: "%" }
    - { name: "root", host: "localhost" }
  become: yes
  run_once: true
  delegate_to: "{{ groups['galera'][0] }}"
- name: "USERS: Setup Replication User for Async Node"
  community.mysql.mysql_user:
    name: "{{ repl_user }}"
    password: "{{ repl_password }}"
    host: "%"
    priv: "*.*:REPLICATION SLAVE"
    state: present
    login_user: root
    login_password: "{{ db_admin_password }}"
    login_unix_socket: /var/run/mysqld/mysqld.sock
  become: yes
  run_once: true
  delegate_to: "{{ groups['galera'][0] }}"
```

Galera Config Jinja file

(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/galera/templates/galera.cnf.j2

`pc.recovery=1` (Primary Component Recovery):

- Normally, if all nodes crash, they lose their "Quorum" (the sense of who is the boss).
- This setting tells the nodes to save their last state to a file (`gvwstate.dat`) on the disk before they die. When they power back up, they read that file to automatically figure out who was in the cluster, allowing the cluster to **self-heal** without you having to manually bootstrap it.

- **`gcache.recover=yes`:**
  - The `gcache` is the circular buffer that handles **IST** (Incremental Sync).
  - Without this, if a node restarts, it wipes its cache and is forced to do a heavy **SST** (full data copy). With this enabled, it tries to recover the cache from the disk, allowing it to perform a fast **IST** instead.

2. wsrep_retry_autocommit = 3

This is the **"Conflict Resolution"** setting.

- In a multi-master cluster like Galera, two different nodes might try to update the exact same row at the exact same millisecond. This is called a **certification conflict**.
- Instead of immediately sending an "Error 1213: Deadlock" back to your application, this parameter tells MariaDB: *"If a conflict happens, don't give up yet. Automatically try to commit the transaction again up to **3 times**."*
- **Result:** This makes your cluster feel much more stable to the end-user/application by hiding minor "traffic jams" in the data replication.

```
None
[mysqld]
# --- Core Galera Settings ---
bind-address                = {{ bind_address }}
default_storage_engine      = {{ default_storage_engine }}
binlog_format               = {{ binlog_format }}
innodb_autoinc_lock_mode    = {{ innodb_autoinc_lock_mode }}
# --- Cluster Identity ---
wsrep_on                    = {{ wsrep_on }}
wsrep_provider              = {{ wsrep_provider }}
wsrep_cluster_name          = "{{ wsrep_cluster_name }}"
wsrep_cluster_address       = "{{ wsrep_cluster_address }}"
wsrep_node_name             = "{{ inventory_hostname }}"
```

```
wsrep_node_address          = "{{ ansible_host }}"
# --- State Snapshot Transfer (SST) ---
wsrep_sst_method            = {{ wsrep_sst_method }}
wsrep_sst_auth              = "{{ wsrep_sst_auth }}"
# --- Async Replication Support (The "Publisher" Config) ---
log_bin                      = /var/log/mysql/mariadb-bin
log_slave_updates            = 1  # Vital: Write Galera changes to the binlog
max_binlog_size              = 100M
# --- Version Specific Logic ---
{# Targets: 10.6.16 and 10.6.21 #}
{% if mariadb_version is version('10.6', '>=') and mariadb_version is version('10.7', '<')
%}
# Configuration for MariaDB 10.6.x (Detected: {{ mariadb_version }})
wsrep_mode                   = {{ wsrep_mode | default('REQUIRED_PRIMARY_VIEW') }}
binlog_expire_logs_seconds   = {{ binlog_expire_logs_seconds | default(604800) }}
innodb_buffer_pool_instances = {{ innodb_buffer_pool_instances | default(1) }}
{% endif %}
{# Targets: 10.11.16 #}
{% if mariadb_version is version('10.11', '>=') and mariadb_version is version('11.0',
'<') %}
# Configuration for MariaDB 10.11.x LTS (Detected: {{ mariadb_version }})
wsrep_mode                   = {{ wsrep_mode | default('STRICT_REPLICATION') }}
binlog_expire_logs_seconds   = {{ binlog_expire_logs_seconds | default(604800) }}
innodb_buffer_pool_instances = {{ innodb_buffer_pool_instances | default(1) }}
wsrep_slave_threads          = {{ wsrep_slave_threads | default(4) }}
innodb_undo_tablespaces      = {{ innodb_undo_tablespaces | default(3) }}
wsrep_gtid_domain_id         = {{ wsrep_gtid_domain_id | default(1) }}
# LTS Stability Tweaks
wsrep_provider_options       = "pc.recovery=1;gcache.recover=yes"
wsrep_retry_autocommit       = 3
{% endif %}
```

HCIP Setup in galera

(venv) ubuntu@head-node:~/harbour-master/backend$ sudo cat ansible/roles/galera/templates/ db_keepalived.conf.j2

```
None
vrrp_script chk_galera {
    script "/usr/local/sbin/galera_healthcheck.sh"
    interval 2   #runs script every 2 sec
    timeout 2    #longer than 2 secon then failure
    fall 3       #3 consecutive failure allowed
    rise 2  #requires 2 consecutive success
}
vrrp_instance GHC1 {
```

```
    state BACKUP
    interface {{ ansible_default_ipv4.interface }}  #primary network interface
    virtual_router_id 51     #unique cluster id
    # Higher priority for db-01 so it wins ONLY the first time or if both are fresh
    priority {{ 110 if inventory_hostname == groups['galera'][0] else 100 }}
    advert_int 1  #sends heartbeat everyone second

    # Critical: Do not grab the IP back if the other node is currently Master
    nopreempt
    unicast_src_ip {{ ansible_default_ipv4.address }}
    unicast_peer {
        {% for host in groups['galera'][:2] %}
        {% if host != inventory_hostname %}
        {{ hostvars[host]['ansible_default_ipv4']['address'] }}
        {% endif %}
        {% endfor %}
    }
    virtual_ipaddress {
        {{ hcip_value }}/32 dev dummy0
    }
    track_script {
        chk_galera     # If script fails, this instance gives up the VIP
    }
}
```

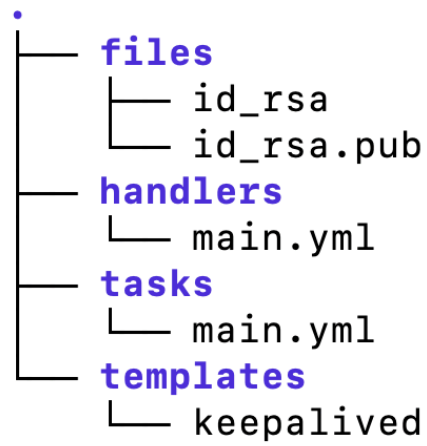
(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/galera/templates/galera_hcip_manager.sh.j2

```
None
#!/bin/bash
PASS="{{ db_admin_password }}"
# 1. Check if MariaDB is responding
if ! mariadb -u root -p"$PASS" --connect-timeout=2 -e "status" >/dev/null 2>&1; then
    exit 1
fi
# 2. Check Galera Cluster Status
# We only care if the node is 'Synced'.
STATE=$(mariadb -u root -p"$PASS" -N -s -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"
| awk '{print $2}')
if [ "$STATE" == "Synced" ]; then
    exit 0
else
    exit 1
fi
```

(venv) ubuntu@head-node:~/harbour-master/backend$ sudo cat ansible/roles/galera/handlers/main.yml
Systemd service remove

```
None
---
- name: Restart SSH
  service:
    name: ssh
    state: restarted
  become: yes
- name: Restart Keepalived
  service:
    name: keepalived
    state: restarted
  become: yes
- name: Restart HCIP Service
  debug:
    msg: "Legacy HCIP Service Handler - No longer used"
```

LVS Setup



Keepalived and Health check through dummy0 Interface

(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/lvs/handlers/main.yml

```
None
---
- name: Restart Keepalived
  service:
    name: keepalived
    state: restarted
```

(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/lvs/tasks/main.yml

```
None
---
# 1. Install Keepalived and Ipvsadm
- name: Install LVS and Keepalived dependencies
  apt:
    name:
      - keepalived
      - ipvsadm
    state: present
    update_cache: yes
  become: yes
# 2. Enable IP Forwarding
```

```
- name: Enable IPv4 forwarding
  sysctl:
    name: net.ipv4.ip_forward
    value: '1'
    state: present
    reload: yes
  become: yes
# 3. Setup SSH for Health Checks
- name: Ensure .ssh directory exists for root
  file:
    path: /root/.ssh
    state: directory
    mode: '0700'
  become: yes
- name: Deploy SSH Private Key for LVS Health Checks
  copy:
    src: id_rsa
    dest: /root/.ssh/id_rsa
    owner: root
    group: root
    mode: '0600'
  become: yes
# 4. Deploy Health Check Script
# FIX: Using 'template' style or escaping the $1 is better to prevent Ansible from
misinterpreting it.
- name: Deploy LVS Health Check script
  copy:
    content: |
      #!/bin/bash
      # Usage: ./check_hcip_ssh.sh <DB_IP>
      # Check if dummy0 has the HCIP via SSH
      # Added -o BatchMode=yes to ensure it never hangs waiting for a password
      ssh -q -o ConnectTimeout=2 \
          -o StrictHostKeyChecking=no \
          -o UserKnownHostsFile=/dev/null \
          -o BatchMode=yes \
          root@$1 "ip addr show dummy0 | grep -q {{ hcip_value }}"
    dest: /usr/local/bin/check_hcip_ssh.sh
    mode: '0755'
  become: yes
# 5. Deploy Keepalived Configuration
- name: Deploy Keepalived configuration from template
  template:
    src: keepalived.conf.j2
    dest: /etc/keepalived/keepalived.conf
    owner: root
    group: root
    mode: '0644'
  become: yes
  notify: Restart Keepalived
```

```
# 6. Ensure Keepalived is started
- name: Start and Enable Keepalived
  service:
    name: keepalived
    state: started
    enabled: yes
  become: yes
```

Keepalived Config Jinja

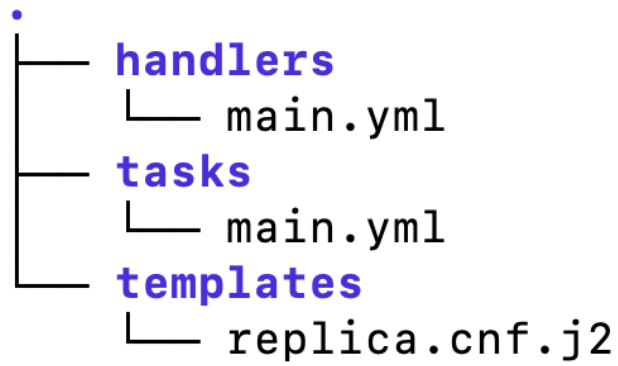
(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/lvs/templates/keepalived.conf.j2

```
None
global_defs {
    router_id {{ inventory_hostname }}  #lvs host name
    enable_script_security  //security scripts checks permission
    script_user root  //checks health script as the root user
}
vrrp_instance VI_1 {
    state MASTER
    interface {{ ansible_default_ipv4.interface }}
    virtual_router_id 50
    priority 100
    advert_int 1 sends heartbeat to peer every 1 sec
    nopreempt
    unicast_src_ip {{ ansible_default_ipv4.address }}
    unicast_peer {
        {% for host in groups['lvs'] %}
        {% if host != inventory_hostname %}
        {{ hostvars[host]['ansible_default_ipv4']['address'] }}
        {% endif %}
        {% endfor %}
    }
    virtual_ipaddress {
        {{ lvs_vip }}
    }
}
virtual_server {{ lvs_vip }} 3306 {
    delay_loop 2   # wait 2 sec between health checks
    lb_algo rr
```

```
    lb_kind DR
    protocol TCP
    # Dynamically generate real_server entries for all Galera nodes
    {% for host in groups['galera'] %}
    # We only include the first two nodes as potential writers per your setup
    {% if loop.index <= 2 %}
    real_server {{ hostvars[host]['ansible_default_ipv4']['address'] }} 3306 {
        weight 1
        MISC_CHECK {
            # We pass the physical node IP to the script, which checks for the HCIP
            misc_path "/usr/local/bin/check_hcip_ssh.sh {{
hostvars[host]['ansible_default_ipv4']['address'] }}"
            misc_timeout 5  # if takes more than 5sec then down
            user root
        }
    }
    {% endif %}
    {% endfor %}
}
```

(venv) ubuntu@head-node:~/harbor-master$ mkdir -p ~/harbor-master/env

Async Setup



(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/async_replica/handlers/main.yml

```
None
---
- name: Restart MariaDB
  service:
    name: mariadb
    state: restarted
```

(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/lvs/templates/async_replica/tasks/main.yml

This Ansible playbook automates the setup of an **Asynchronous Read Replica** connected to a primary Galera Cluster. It begins by identifying a healthy Galera master (specifically targeting `db-03`) and verifying if replication is already running to avoid redundant configurations. If the replica is not healthy, the playbook performs a "nuclear reset" of the replication state to ensure a clean slate, synchronizes the **Global Transaction ID (GTID)** position between the master and the slave, and establishes a secure handshake using a dedicated replication user. By dynamically aligning the slave's position with the master's binlog and applying specialized MariaDB configurations, the playbook ensures that the async node can offload read traffic from the main cluster while maintaining data consistency

```
None
---
# 0. PRE-REQUISITE: INSTALL & CLUSTER READINESS
- name: Ensure MariaDB Server is installed on Async Node
  apt:
    name:
      - mariadb-server
      - python3-pymysql
    state: present
    update_cache: yes
  become: yes

- name: Identify Healthy Galera Nodes
  shell: "mariadb -u root -p'{{ db_admin_password }}' -N -s -e \"SHOW STATUS LIKE
'wsrep_local_state_comment';\""
  become: yes
  delegate_to: "{{ item }}"
  loop: "{{ groups['galera'] }}"
  register: galera_health_results
  ignore_errors: yes
  changed_when: false
```

```
- name: Set Active Galera Master Fact (Force db-03)
  set_fact:
    # This directly picks the 3rd node in the 'galera' inventory group
    active_galera_master: "{{ groups['galera'][2] }}"
  run_once: true

- name: Check current Replication Status on Async Node
  shell: "mariadb -u root -p'{{ db_admin_password }}' -e 'SHOW REPLICA STATUS\\G'"
  register: slave_status_raw
  ignore_errors: yes
  become: yes
  changed_when: false
- name: Set Replication Health Fact
  set_fact:
    is_replication_healthy: "{{ 'Slave_IO_Running: Yes' in slave_status_raw.stdout and
'Slave_SQL_Running: Yes' in slave_status_raw.stdout }}"
  when: slave_status_raw.rc == 0

- name: Ensure Replication User has access from Async Node IP
  community.mysql.mysql_user:
    name: "{{ repl_user }}"
    password: "{{ repl_password }}"
    host: "{{ ansible_host }}"
    priv: "*.*:REPLICATION SLAVE"
    state: present
    login_user: root
    login_password: "{{ db_admin_password }}"
    login_unix_socket: /var/run/mysqld/mysqld.sock
    check_implicit_admin: yes
  become: yes
  delegate_to: "{{ active_galera_master }}"
  run_once: true
  when: not (is_replication_healthy | default(false))

- name: Ensure MariaDB config directory exists
  file:
    path: /etc/mysql/mariadb.conf.d
    state: directory
    owner: root
    group: root
    mode: '0755'
  become: yes
- name: Apply Standalone Replica Configuration
  template:
    src: replica.cnf.j2
    dest: /etc/mysql/mariadb.conf.d/70-replica.cnf
  become: yes
  register: replica_conf
- name: Grant Control Node access to Async Node for Diagnostics
```

```
  community.mysql.mysql_user:
    name: root
    password: "{{ db_admin_password }}"
    # Robust IP detection: Try SSH_CLIENT, then Control Node IP, then fallback to Head
Node's specific subnet IP
    host: "{{ (ansible_env.SSH_CLIENT | default('')).split() | first |
default(hostvars['localhost']['ansible_default_ipv4']['address'] |
default('192.168.64.191')) }}"
    priv: "*.*:ALL,GRANT"
    state: present
    login_user: root
    login_password: "{{ db_admin_password }}"
    login_unix_socket: /var/run/mysqld/mysqld.sock
  become: yes
- name: Restart MariaDB to apply configs
  systemd:
    name: mariadb
    state: restarted
  become: yes
  when: replica_conf.changed
# 3. THE "NUCLEAR" RESET
- name: Hard Reset Slave State for GTID alignment
  shell: |
    mariadb -u root -p'{{ db_admin_password }}' -e "STOP REPLICA;"
    mariadb -u root -p'{{ db_admin_password }}' -e "RESET REPLICA ALL;"
    mariadb -u root -p'{{ db_admin_password }}' -e "SET GLOBAL gtid_slave_pos = '';"
  become: yes
  when: not (is_replication_healthy | default(false))
# 4. CAPTURE MASTER POSITION
- name: Get Master GTID Position
  shell: "mariadb -u root -p'{{ db_admin_password }}' -N -s -e 'SELECT
@@gtid_binlog_pos;'"
  become: yes
  delegate_to: "{{ active_galera_master }}"
  register: master_gtid
  run_once: true
  when: not (is_replication_healthy | default(false))
# 5. SYNC SLAVE POSITION
- name: Align Slave GTID position
  shell: "mariadb -u root -p'{{ db_admin_password }}' -e \"SET GLOBAL gtid_slave_pos = '{{
master_gtid.stdout | trim }}';\""
  become: yes
  when:
    - not (is_replication_healthy | default(false))
    - master_gtid is defined and master_gtid.rc == 0
# 6. CONFIGURE AND START
- name: Configure Master Handshake
  community.mysql.mysql_replication:
    mode: changeprimary
    # Connect the slave specifically to the healthy master we found
```

```
    primary_host: "{{ hostvars[active_galera_master]['ansible_host'] }}"
    primary_user: "{{ repl_user }}"
    primary_password: "{{ repl_password }}"
    primary_use_gtid: replica_pos
    login_user: root
    login_password: "{{ db_admin_password }}"
    login_unix_socket: /var/run/mysqld/mysqld.sock
  become: yes
  when: not (is_replication_healthy | default(false))
- name: Start Replication
  community.mysql.mysql_replication:
    mode: startreplica
    login_user: root
    login_password: "{{ db_admin_password }}"
    login_unix_socket: /var/run/mysqld/mysqld.sock
  become: yes
  when: not (is_replication_healthy | default(false))
# 7. FINAL VERIFICATION
- name: Verify Replication Status
  shell: "mariadb -u root -p'{{ db_admin_password }}' -e 'SHOW REPLICA STATUS\\G'"
  register: final_check
  become: yes
  until: '"Slave_IO_Running: Yes" in final_check.stdout and "Slave_SQL_Running: Yes" in
final_check.stdout'
  retries: 15
  delay: 10
```

Async Config Jinja

(venv) ubuntu@head-node:~/harbor-master$ sudo vim ansible/roles/lvs/templates/async_replica/templates/replica.cnf.j2

```
None
[mysqld]
# --- Core Networking ---
bind-address            = 0.0.0.0
# --- Unique Identity ---
# Deterministic ID: (3rd_octet * 256) + 4th_octet
{% set octets = ansible_host.split('.') %}
server-id               = {{ (octets[2]|int * 256) + octets[3]|int }}
report_host             = {{ inventory_hostname }}
# --- Replication Settings ---
```

```
gtid_domain_id          = {{ wsrep_gtid_domain_id | default(1) }}
log_bin                 = /var/log/mysql/mariadb-bin
binlog_format           = ROW
# --- Safety & Performance ---
read_only               = 1
innodb_autoinc_lock_mode = 2
# --- Disable Galera ---
wsrep_on                = OFF
wsrep_provider          = none
# --- Logging Management ---
expire_logs_days        = {{ expire_logs_days | default(7) }}
max_binlog_size         = 100M
# 1396: Operation CREATE USER failed
# 1062: Duplicate entry for primary key
slave-skip-errors       = 1396, 1062
```

Monitoring Setup

```
.
├── defaults
│   └── main.yml
├── files
│   └── dashboards
│       ├── mariadb_deep_dive.json
│       └── system_metrics.json
├── handlers
│   └── main.yml
├── tasks
│   ├── exporters.yml
│   ├── main.yml
│   └── server.yml
└── templates
    ├── dashboards_provider.yml.j2
    ├── datasources.yml.j2
    ├── galera_dashboard.json.j2
    ├── grafana_alerts.yaml.j2
    ├── grafana_contact_points.yaml.j2
    ├── grafana_policies.yaml.j2
    └── prometheus.yml.j2
```

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring**$ mkdir -p defaults
(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/defaults**$ sudo vim main.yml

```
None
---
# The password for the MariaDB 'exporter' user
exporter_password: "monitoring_password_2026"
```

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring**$ mkdir -p files/dashboards
(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring**$ cd files
(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/files**$ cd dashboards/

(venv)
ubuntu@head-node:**~/harbor-master/ansible/roles/monitoring/files/dashboards**$ curl -sL https://grafana.com/api/dashboards/1860/revisions/37/download -o system_metrics.json
(venv)
ubuntu@head-node:**~/harbor-master/ansible/roles/monitoring/files/dashboards**$ curl -sL https://grafana.com/api/dashboards/14057/revisions/1/download -o mariadb_deep_dive.json
Made files such that they are minimal from the above template
(venv)
ubuntu@head-node:**~/harbour-master/backend/ansible/roles/monitoring/files/dashboards**$ ls
mariadb_deep_dive.json system_metrics.json

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring**$ mkdir -p handlers

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/handlers**$ sudo vim main.yml

```
None
---
- name: Restart Prometheus
  systemd:
    name: prometheus
    state: restarted
    daemon_reload: yes
- name: Restart Grafana
  systemd:
    name: grafana-server
    state: restarted
    daemon_reload: yes
- name: Restart MySQL Exporter
  systemd:
    name: prometheus-mysqld-exporter
    state: restarted
- name: restart node exporter
  become: true
  systemd:
    name: prometheus-node-exporter
    state: restarted
    daemon_reload: true
```

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring**$ mkdir -p tasks

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/handlers**$ sudo vim exporters.yml

```
None
---
# 1. INSTALLATION
- name: Install Node and MySQL Exporters
  apt:
    name:
      - prometheus-node-exporter
      - prometheus-mysqld-exporter
      - python3-pymysql
    state: present
    update_cache: yes
  become: yes

- name: "Create MariaDB Exporter User"
  community.mysql.mysql_user:
    name: "exporter"
    password: "{{ exporter_password }}"
    priv: "*.*:PROCESS,REPLICATION CLIENT,SELECT"
    state: present
    login_user: root
    login_password: "{{ db_admin_password }}"
    login_unix_socket: /var/run/mysqld/mysqld.sock
    check_implicit_admin: yes
  become: yes

  register: exporter_user_result
  until: exporter_user_result is not failed
  retries: 10
  delay: 5
  when: inventory_hostname in (groups['galera'] + groups['async'] | default([]))

# 4. CONFIGURATION (IDEMPOTENT)
- name: Configure MySQL Exporter Auth
  copy:
    dest: /etc/mysql/exporter.cnf
    owner: prometheus
    group: prometheus
    mode: '0400'
    content: |
      [client]
      user=exporter
      password={{ exporter_password }}
      host=localhost
```

```
  become: yes
  register: exporter_cfg
  when: inventory_hostname in (groups['galera'] + groups['async'] | default([]))

- name: Set MySQL Exporter Flags
  lineinfile:
    path: /etc/default/prometheus-mysqld-exporter
    regexp: '^ARGS='
    line: 'ARGS="--config.my-cnf=/etc/mysql/exporter.cnf --web.listen-address=:9104"'
  become: yes
  register: exporter_flags
  when: inventory_hostname in (groups['galera'] + groups['async'] | default([]))

- name: Set Node Exporter Flags
  lineinfile:
    path: /etc/default/prometheus-node-exporter
    regexp: '^ARGS='
    # ADDED --collector.network_route below
    line: 'ARGS="--collector.netdev.address-info --collector.ipvs
--collector.network_route"'
  become: yes
  notify: restart node exporter

# 5. SERVICE MANAGEMENT
- name: Ensure Monitoring Agents are started and enabled
  systemd:
    name: "{{ item }}"
    state: "{{ 'restarted' if (exporter_cfg.changed or exporter_flags.changed) else
'started' }}"
    enabled: yes
    daemon_reload: yes
  become: yes
  loop:
    - prometheus-node-exporter
    - prometheus-mysqld-exporter
  when: inventory_hostname in (groups['galera'] + groups['async'] | default([]))
- name: Ensure Node Exporter is active on non-DB nodes
  systemd:
    name: prometheus-node-exporter
    state: started
    enabled: yes
  become: yes
  when: inventory_hostname not in (groups['galera'] + groups['async'] | default([]))
```

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/handlers**$ sudo vim main.yml

```
None
- name: "[SERVER] Deploy Prometheus and Grafana Engine"
  include_tasks: server.yml
  # This ensures the heavy monitoring stack only installs on your management node
  when: inventory_hostname == 'monitor-node' or inventory_hostname in groups['monitoring']
- name: "[CLIENT] Deploy Metrics Exporters"
  include_tasks: exporters.yml
  # This deploys Node Exporter and MySQL Exporter to all relevant targets
  # We skip the monitor-node for the MySQL exporter logic specifically inside
exporters.yml
  when: inventory_hostname != 'monitor-node'
```

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/handlers**$ sudo vim server.yml

```
None
---
- name: Install dependencies for Grafana
  apt:
    name: [apt-transport-https, software-properties-common, wget, gpg]
    state: present
  become: yes
- name: Create directory for APT keyrings
  file:
    path: /etc/apt/keyrings
    state: directory
    mode: '0755'
  become: yes
- name: Import Grafana GPG key
  get_url:
    url: https://apt.grafana.com/gpg.key
    dest: /etc/apt/keyrings/grafana.asc
    mode: '0644'
  become: yes
- name: Add Grafana official APT repository
  shell: |
    echo "deb [signed-by=/etc/apt/keyrings/grafana.asc] https://apt.grafana.com stable
main" > /etc/apt/sources.list.d/grafana.list
  become: yes
- name: Update apt cache and Install Monitoring Stack
  apt:
    name: [prometheus, grafana]
    state: present
```

```
    update_cache: yes
  become: yes
- name: Create Provisioning Directories
  file:
    path: "{{ item }}"
    state: directory
    owner: grafana
    group: grafana
    mode: '0755'
  loop:
    - /etc/grafana/provisioning/datasources
    - /etc/grafana/provisioning/dashboards
    - /etc/grafana/provisioning/alerting
    - /var/lib/grafana/dashboards
  become: yes
- name: Deploy Prometheus Configuration
  template:
    src: prometheus.yml.j2
    dest: /etc/prometheus/prometheus.yml
    owner: prometheus
    group: prometheus
    mode: '0644'
  become: yes
  notify: Restart Prometheus
# --- ALERTING PROVISIONING ---
- name: Provision Grafana Alert Rules
  template:
    src: grafana_alerts.yaml.j2
    dest: /etc/grafana/provisioning/alerting/harbor_alerts.yaml
    owner: root
    group: grafana
    mode: '0640'
  become: yes
  notify: Restart Grafana
- name: Deploy Grafana Contact Points
  template:
    src: grafana_contact_points.yaml.j2
    dest: /etc/grafana/provisioning/alerting/contact_points.yaml
    owner: root
    group: grafana
    mode: '0640'
  become: yes
  notify: Restart Grafana
- name: Deploy Grafana Notification Policies
  template:
    src: grafana_policies.yaml.j2
    dest: /etc/grafana/provisioning/alerting/policies.yaml
    owner: root
    group: grafana
    mode: '0640'
```

```
  become: yes
  notify: Restart Grafana
# --- SMTP AND SYSTEM CONFIG ---
- name: Configure Grafana SMTP settings
  ini_file:
    path: /etc/grafana/grafana.ini
    section: smtp
    option: "{{ item.option }}"
    value: "{{ item.value }}"
    owner: root
    group: grafana
    mode: '0640'
  loop:
    - { option: 'enabled', value: 'true' }
    - { option: 'host', value: 'smtp.gmail.com:587' }
    - { option: 'user', value: 'shambhavi.intern@phonepe.com' }
    - { option: 'password', value: 'iruistkvxgkxunyf' }
    - { option: 'from_address', value: 'shambhavi.intern@phonepe.com' }
    - { option: 'startTLS_policy', value: 'MandatoryStartTLS' }
  become: yes
  no_log: true
  notify: Restart Grafana
# --- DASHBOARDS AND DATASOURCES ---
- name: Provision Data Source (Prometheus)
  template:
    src: datasources.yml.j2
    dest: /etc/grafana/provisioning/datasources/prometheus.yaml
    owner: root
    group: grafana
    mode: '0640'
  become: yes
  notify: Restart Grafana
- name: Sync Dashboard JSON Files
  copy:
    src: dashboards/
    dest: /var/lib/grafana/dashboards/
    owner: grafana
    group: grafana
    mode: '0644'
  become: yes
  notify: Restart Grafana
- name: Provision Dashboard Provider
  template:
    src: dashboards_provider.yml.j2
    dest: /etc/grafana/provisioning/dashboards/provider.yaml
    owner: root
    group: grafana
    mode: '0640'
  become: yes
  notify: Restart Grafana
```

```
- name: Render Custom Galera Dashboard
  template:
    src: galera_dashboard.json.j2
    dest: /var/lib/grafana/dashboards/galera_dashboard.json
    owner: grafana
    group: grafana
    mode: '0644'
  become: yes
  notify: Restart Grafana
- name: Start and Enable Services
  systemd:
    name: "{{ item }}"
    state: started
    enabled: yes
    daemon_reload: yes
  loop:
    - prometheus
    - grafana-server
  become: yes
```

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring**$ mkdir -p templates

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/templates**$ sudo vim dashboards_provider.yml.j2

Instead of us manually clicking "Import" in the web interface, Grafana reads this file at startup and builds the UI automatically.

```
None
apiVersion: 1
providers:
  - name: 'Cluster Monitoring'
    orgId: 1
    folder: 'Database'
    type: file
    disableDeletion: false
    editable: true
    options:
```

```
      path: /var/lib/grafana/dashboards
    translations:
      - name: DS_PROMETHEUS
        uid: "prom_ds_phase5
```

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/templates**$ sudo vim datasources.yml.j2

"The `datasources.yml` file is our **Automation Bridge**. It programmatically connects Grafana to our Prometheus time-series database. By using a **Static UID (`prom_ds_phase5`)**, we create a permanent link between our data source and our custom dashboards. This means the moment the stack is deployed, the charts are already 'Live' and pulling data, fulfilling our goal of a zero-touch, automated monitoring environment."

```
None
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    url: http://localhost:9090
    uid: "prom_ds_phase5"  # Fixed UID for dashboard linking
    isDefault: true
    editable: true
```

(venv) ubuntu@head-node:~/harbor-master/ansible/roles/monitoring/templates$ sudo vim galera_dashboard.json.j2 - has overview dashboard json

(venv) **ubuntu@head-node**:**~/harbour-master/backend/ansible/roles/monitoring/templates**$ sudo cat grafana_alerts.yaml.j2

```
None
apiVersion: 1
groups:
  - orgId: 1
    name: HarborMasterAlerts
    folder: Infrastructure
    interval: 10s
```

```
    rules:
      # 1. GALERA CLUSTER SIZE
      - uid: galera_shrunk
        title: Galera Cluster Size Shrunk
        condition: C
        data:
          - refId: A
            datasourceUid: "prom_ds_phase5"
            relativeTimeRange: { from: 300, to: 0 }
            model: { expr:
"max(mysql_global_status_wsrep_cluster_size{instance=~'db-.*'})" }
          - refId: B
            datasourceUid: "__expr__"
            model: { type: "reduce", expression: "A", reducer: "last", settings: { mode:
"dropNN" } }
          - refId: C
            datasourceUid: "__expr__"
            model: { type: "threshold", expression: "B", conditions: [{ evaluator: {
params: [3], type: "lt" }, operator: { type: "and" }, query: { params: [] }, type: "query"
}] }
        for: 0s
        annotations: { summary: "Galera Cluster Shrunk", description: "Cluster size below
3 nodes. Quorum at risk!" }
        labels: { severity: critical }
      # 3. ASYNC NODE DOWN (OS Level)
      - uid: async_node_down
        title: Async Node Down
        condition: C
        data:
          - refId: A
            datasourceUid: "prom_ds_phase5"
            relativeTimeRange: { from: 60, to: 0 }
            model: { expr: "up{instance=~'.*async.*'}" }
          - refId: B
            datasourceUid: "__expr__"
            model: { type: "reduce", expression: "A", reducer: "last", settings: { mode:
"dropNN" } }
          - refId: C
            datasourceUid: "__expr__"
            model: { type: "threshold", expression: "B", conditions: [{ evaluator: {
params: [0], type: "eq" }, operator: { type: "and" }, query: { params: [] }, type: "query"
}] }
        for: 0s
        annotations: { summary: "Async Node Offline", description: "Prometheus cannot
reach the async node exporter." }
        labels: { severity: critical }
      # 4. ASYNC MARIADB STOPPED
      - uid: async_mariadb_stopped
        title: Async MariaDB Stopped
        condition: C
```

```
        data:
          - refId: A
            datasourceUid: "prom_ds_phase5"
            relativeTimeRange: { from: 60, to: 0 }
            model: { expr: "absent(mysql_version_info{instance=~'.*async.*'}) or on()
vector(0)" }
          - refId: B
            datasourceUid: "__expr__"
            model: { type: "reduce", expression: "A", reducer: "last", settings: { mode:
"dropNN" } }
          - refId: C
            datasourceUid: "__expr__"
            model: { type: "threshold", expression: "B", conditions: [{ evaluator: {
params: [1], type: "eq" }, operator: { type: "and" }, query: { params: [] }, type: "query"
}] }
        for: 0s
        annotations: { summary: "Async MariaDB Stopped", description: "MariaDB process is
down on the async node." }
        labels: { severity: critical }

      - uid: hcip_offline
        title: HCIP_Offline
        condition: C
        data:
          - refId: A
            datasourceUid: "prom_ds_phase5"
            relativeTimeRange: { from: 60, to: 0 }
            # Filter specifically for Galera nodes to detect if the Cluster IP is missing
            model: { expr: "sum(node_network_address_info{address='10.0.0.100',
instance=~'db-.*'}) or vector(0)" }
          - refId: B
            datasourceUid: "__expr__"
            model: { type: "reduce", expression: "A", reducer: "last", settings: { mode:
"dropNN" } }
          - refId: C
            datasourceUid: "__expr__"
            model: { type: "threshold", expression: "B", conditions: [{ evaluator: {
params: [1], type: "lt" }, operator: { type: "and" }, query: { params: [] }, type: "query"
}] }
        for: 30s # Adding a small buffer to prevent false alarms during quick failovers
        annotations: { summary: "HCIP Offline", description: "The HCIP ON dummy O is
missing from both DB nodes!" }
        labels: { severity: critical }
      - uid: vip_offline
        title: VIP_Offline
        condition: C
        data:
          - refId: A
            datasourceUid: "prom_ds_phase5"
            relativeTimeRange: { from: 60, to: 0 }
```

```
            # Filter specifically for LVS nodes so Galera's local VIP doesn't trick us
            model: { expr: "sum(node_network_address_info{address='{{ lvs_vip }}',
instance=~'lvs-.*'}) or vector(0)" }
          - refId: B
            datasourceUid: "__expr__"
            model: { type: "reduce", expression: "A", reducer: "last", settings: { mode:
"dropNN" } }
          - refId: C
            datasourceUid: "__expr__"
            model: { type: "threshold", expression: "B", conditions: [{ evaluator: {
params: [1], type: "lt" }, operator: { type: "and" }, query: { params: [] }, type: "query"
}] }
        for: 0s
        annotations: { summary: "VIP Offline", description: "LVS VIP is missing from all
Load Balancers!" }
        labels: { severity: critical }
```

(venv) **ubuntu@head-node**:**~/harbour-master/backend/ansible/roles/monitoring/templates**$ sudo cat grafana_contact_points.yaml.j2

```
None
apiVersion: 1
contactPoints:
  - orgId: 1
    name: Email_Admin
    receivers:
      - uid: email_receiver_1
        type: email
        disableResolveMessage: false
        settings:
          addresses: "shambhavi.intern@phonepe.com"
          singleEmail: true
```

(venv) **ubuntu@head-node**:**~/harbour-master/backend/ansible/roles/monitoring/templates**$ sudo cat grafana_policies.yaml.j2

None

```
apiVersion: 1
policies:
  - orgId: 1
    receiver: Email_Admin
    group_by: ['alertname', 'severity']
    group_wait: 5s
    group_interval: 10s
    repeat_interval: 1h
    # The root policy above MUST NOT have matchers.
    # We define the specific rules in the 'routes' list below.
    routes:
      - receiver: Email_Admin
        group_wait: 5s
        group_interval: 10s
        matchers:
          - "severity =~ critical|warning"
        continue: true
```

(venv) **ubuntu@head-node**:**~/harbor-master/ansible/roles/monitoring/templates**$ sudo vim prometheus.yml.j2

None

```
global:
  scrape_interval: 5s
  evaluation_interval: 5s  # How often to check the alert rules
scrape_configs:
  - job_name: 'node_exporter'
    static_configs:
{% for host in groups['all'] %}
      - targets: ['{{ hostvars[host].ansible_host |
default(hostvars[host].inventory_hostname) }}:9100']
        labels:
          nodename: '{{ host }}' # Use a temporary label
{% endfor %}
    relabel_configs:
      - source_labels: [nodename]
        target_label: instance # Overwrite the default IP:Port instance label
  - job_name: 'mysql'
    static_configs:
{% for host in (groups['galera'] | default([]) + groups['async'] | default([])) | unique
%}
      - targets: ['{{ hostvars[host].ansible_host |
default(hostvars[host].inventory_hostname) }}:9104']
```

```
        labels:
          nodename: '{{ host }}'
{% endfor %}
    relabel_configs:
      - source_labels: [nodename]
        target_label: instance
```

(venv) **ubuntu@head-node**:**~/harbour-master/backend**$ sudo cat Dockerfile

```
None
# STAGE 1: Builder (Compiles C-extensions)
FROM python:3.12-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y \
    gcc \
    python3-dev \
    libssl-dev \
    libffi-dev \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
# We build wheels for heavy items like cryptography/paramiko here
RUN pip wheel --no-cache-dir --wheel-dir /app/wheels -r requirements.txt
# STAGE 2: Final (The production image)
FROM python:3.12-slim
WORKDIR /app
# 1. Install runtime OS dependencies
RUN apt-get update && apt-get install -y \
    openssh-client \
    iputils-ping \
    && rm -rf /var/lib/apt/lists/*
# 2. Copy and install pre-built wheels from builder
COPY --from=builder /app/wheels /wheels
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels
# 3. Install Ansible & MySQL support in ONE layer to save space
RUN pip install --no-cache-dir ansible-core pymysql && \
    ansible-galaxy collection install \
    community.crypto \
    ansible.posix \
    community.mysql \
    community.general
COPY . .
EXPOSE 8000
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

(venv) ubuntu@head-node:~/harbor-master$ uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload

Frontend setup

(venv) ubuntu@head-node-test:~/harbor-master/cd ~/harbor-master

(venv) ubuntu@head-node-test:~/harbor-master/npx create-react-app frontend

It generates a standardized folder structure, installs the core React libraries, and sets up a local development server.

(venv) ubuntu@head-node-test:~/harbor-master/
cd frontend

(venv) ubuntu@head-node-test:~/harbor-master/frontend/
npm install lucide-react
Lucide is a library of clean, professional-looking icons (like servers, shields, or checkmarks).
(venv) ubuntu@head-node-test:~/harbor-master/frontend/
vim src/index.js
env

```
None
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
i
```

(venv) ubuntu@head-node-test:~/harbor-master/frontend/
mkdir -p public && vim public/index.html

```
None
<!DOCTYPE html>
```

```
<html lang="en">
  <head><title>Harbor Master</title></head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

(venv) ubuntu@head-node-test:~/harbor-master/frontend/src/App.js

None

```
 import React, { useState, useEffect, useRef } from 'react';
import { HelpCircle, Activity, Server, Database, Loader2, ShieldAlert, CheckCircle,
XCircle, Globe, Zap, Settings, Lock, Eye, EyeOff, ShieldCheck, Layers, Info, Network, Cpu,
Terminal, ArrowRight, ArrowLeft, AlertTriangle, Play, FileCode, Download, User, LogIn }
from 'lucide-react';
function App() {
  // --- AUTHENTICATION STATE ---
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [loginData, setLoginData] = useState({ username: '', password: '' });
  const [loginError, setLoginError] = useState('');
  const [activeTooltip, setActiveTooltip] = useState(null);
  const [isConflictModalOpen, setIsConflictModalOpen] = useState(false);
  const [conflictNodes, setConflictNodes] = useState([]);
  // --- UI & DEPLOYMENT STATE ---
  const [page, setPage] = useState(1); // 1: Config, 2: Deployment/Monitor
  const [loading, setLoading] = useState(false);
  const [validating, setValidating] = useState(false);
  const [report, setReport] = useState(null);
  const [error, setError] = useState(null);
  const [showSSTPass, setShowSSTPass] = useState(false);
  const [showDBAdminPass, setShowDBAdminPass] = useState(false);
  const [showReplPass, setShowReplPass] = useState(false);
  const [previewContent, setPreviewContent] = useState(null);
  const [showPreview, setShowPreview] = useState(false);
  const [progress, setProgress] = useState(0);
  const [milestones, setMilestones] = useState({
    mariadb: "pending",
    galera: "pending",
    lvs: "pending",
    async: "pending",
    monitoring: "pending"
  });
  // --- LOGGING & STREAMING STATE ---
  const [logs, setLogs] = useState([]);
```

```
  const [streamStatus, setStreamStatus] = useState("Disconnected");
  const logContainerRef = useRef(null);
  const [dbForm, setDbForm] = useState({ name: '', user: '', pass: '',ttl:30 });
  const [dbError, setDbError] = useState(null);
  const [dbLoading, setDbLoading] = useState(false);
  const [dbMessage, setDbMessage] = useState(null);
  const [showDbPassword, setShowDbPassword] = useState(false);
  const [userRole, setUserRole] = useState(null);
  const [isAuthModalOpen, setIsAuthModalOpen] = useState(false);
const [authErrorMessage, setAuthErrorMessage] = useState("");
  // --- CONFIGURATION DATA ---
  const [formData, setFormData] = useState({
    mariadb_version: "10.11.16",
    node1_ip: "192.168.64.192", node1_name: "db-node-01",
    node2_ip: "192.168.64.193", node2_name: "db-node-02",
    node3_ip: "192.168.64.194", node3_name: "db-node-03",
    lvs1_ip: "192.168.64.196", lvs2_ip: "192.168.64.197",
    async_ip: "192.168.64.199", monitor_ip: "192.168.64.201",
    lvs_vip: "192.168.64.150",
    wsrep_cluster_name: "Galera_Cluster",
    wsrep_on: "ON",
    wsrep_provider: "/usr/lib/galera/libgalera_smm.so",
    binlog_format: "ROW",
    default_storage_engine: "InnoDB", innodb_autoinc_lock_mode: "2",
    bind_address: "0.0.0.0", wsrep_sst_method: "mariabackup",
    wsrep_sst_auth: "sst_user:password", repl_user: "repl_user",
    repl_password: "ReplicaSecurePass123!",db_admin_password: "AdminSecurePass123!",
    wsrep_strict_ddl: "ON", wsrep_replicate_myisam: "OFF",
    wsrep_mode: "REQUIRED_PRIMARY_KEY,STRICT_REPLICATION", binlog_expire_logs_seconds:
"604800",
    innodb_buffer_pool_instances: "1", wsrep_slave_threads: "4",
    innodb_undo_tablespaces: "3", wsrep_gtid_domain_id: "1"
  });
  // --- AUTHENTICATION HANDLERS ---
  const handleLoginChange = (e) => {
    const { name, value } = e.target;
    setLoginData(prev => ({ ...prev, [name]: value }));
  };
  const handleLoginSubmit = (e) => {
    e.preventDefault();
    if (loginData.username === 'admin' && loginData.password === 'admin') {
    setIsAuthenticated(true);
    setUserRole('admin');
  } else if (loginData.username === 'viewer' && loginData.password === 'viewer') {
    setIsAuthenticated(true);
    setUserRole('viewer');
  } else {
    setLoginError('Invalid credentials.');
  }
  };
```

```
 const parseConflictData = (errorStr) => {
  // Regex to find content inside square brackets [IP: Status, Size: X]
  const regex = /\[(.*?): (.*?), Size: (.*?)\]/g;
  const matches = [];
  let match;
  while ((match = regex.exec(errorStr)) !== null) {
    matches.push({
      ip: match[1],
      status: match[2],
      size: match[3]
    });
  }
  return matches;
};
const ConflictModal = ({ isOpen, data, onClose }) => {
  if (!isOpen) return null;
  return (
    <div style={modalOverlay}>
      <div style={modalContent}>
        <div style={{ color: '#6739B7', marginBottom: '15px' }}>
          <AlertTriangle size={48} style={{ margin: '0 auto' }} />
        </div>
        <h3 style={{ margin: '0 0 10px 0', fontSize: '20px', color: '#512da8', fontWeight:
'700' }}>
          ⚠️ Existing Cluster Detected
        </h3>
        <p style={{ fontSize: '14px', color: '#666', marginBottom: '20px' }}>
          The following nodes already have an active MariaDB installation:
        </p>
        <div style={reportContainer}>
          {data.map((node, index) => (
            <div key={index} style={healthRow}>
              <div style={{ display: 'flex', flexDirection: 'column', alignItems:
'flex-start' }}>
                <span style={{ fontSize: '13px', fontWeight: 'bold' }}>{node.ip}</span>
                <span style={{ fontSize: '11px', color: '#9575cd' }}>Cluster Size:
{node.size}</span>
              </div>
              {/* Using your dynamic badgeStyle function here */}
              <span style={badgeStyle(node.status)}>{node.status}</span>
            </div>
          ))}
        </div>
        <p style={{ fontSize: '12px', color: '#999', marginTop: '15px', fontStyle:
'italic' }}>
          Please wipe these nodes before attempting a new deployment.
        </p>
        <button style={modalBtn} onClick={onClose}>
          Close & Fix
        </button>
```

```
      </div>
    </div>
  );
};
const AuthModal = ({ isOpen, message, onClose }) => {
  if (!isOpen) return null;
  return (
    <div style={modalOverlay}>
      <div style={modalContent}>
        <div style={{ color: '#d32f2f', marginBottom: '15px' }}>
          <ShieldAlert size={48} style={{ margin: '0 auto' }} />
        </div>
        <h3 style={{ margin: '0 0 10px 0', fontSize: '20px', color: '#b71c1c', fontWeight:
'700' }}>
          Access Denied
        </h3>
        <p style={{ fontSize: '15px', color: '#444', lineHeight: '1.6', marginBottom:
'20px' }}>
          {message || "You do not have the required permissions to perform this action."}
        </p>
        <div style={{ backgroundColor: '#fff5f5', padding: '12px', borderRadius: '8px',
border: '1px solid #ffcdd2', fontSize: '13px', color: '#c62828', marginBottom: '20px' }}>
          <strong>Role:</strong> Viewer (Read-Only Access)
        </div>
        <button style={{ ...modalBtn, backgroundColor: '#d32f2f' }} onClick={onClose}>
          Return to Dashboard
        </button>
      </div>
    </div>
  );
};
  // --- SSE LOG STREAMING EFFECT ---
  useEffect(() => {
    let eventSource;
    let reconnectionTimeout;
      const setupSSE = () => {
       if (isAuthenticated && page === 2) {
        console.log("Initiating SSE Connection...");
        eventSource = new EventSource('/api/logs/stream');
        eventSource.onopen = () => {
          setStreamStatus("Connected");
          console.log("SSE Connected Successfully");
        };
        eventSource.onmessage = (event) => {
          const line = event.data;
          // 1. Progress & Milestone Success Logic
          if (line.includes("PHASE: MARIADB_PREP_START")) {
            setMilestones(m => ({...m, mariadb: "loading"}));
            setProgress(15);
          } else if (line.includes("PHASE: GALERA_SETUP_START")) {
```

```
          setMilestones(m => ({...m, mariadb: "done", galera: "loading"}));
          setProgress(40);
        } else if (line.includes("PHASE: LVS_SETUP_START")) {
          setMilestones(m => ({...m, galera: "done", lvs: "loading"}));
          setProgress(65);
        } else if (line.includes("PHASE: ASYNC_SETUP_START")) {
          setMilestones(m => ({...m, lvs: "done", async: "loading"}));
          setProgress(80);
        } else if (line.includes("PHASE: MONITORING_SETUP_START")) {
          setMilestones(m => ({...m, async: "done", monitoring: "loading"}));
          setProgress(90);
        } else if (line.includes("PHASE: DEPLOYMENT_COMPLETE")) {
          setMilestones(m => ({...m, monitoring: "done"}));
          setProgress(100);
        }
        // 2. Error Detection Logic (MOVED INSIDE ONMESSAGE)
        if (line.includes("FAILED!") || line.includes("ERROR!")) {
          setMilestones(m => {
            const newMilestones = { ...m };
            for (let key in newMilestones) {
              if (newMilestones[key] === "loading") {
                newMilestones[key] = "failed";
              }
            }
            return newMilestones;
          });
        }
        // 3. Update the logs
        setLogs((prev) => [...prev, line]);
      }; // <-- onmessage ends here
      eventSource.onerror = (err) => {
        console.error("SSE Error occurred. Attempting to reconnect...");
        setStreamStatus("Disconnected");
        eventSource.close();
      reconnectionTimeout = setTimeout(() => {
          setupSSE();
        }, 3000);
      };
    }
  };
  setupSSE();
return () => {
    if (eventSource) eventSource.close();
    if (reconnectionTimeout) clearTimeout(reconnectionTimeout);
  };
}, [isAuthenticated, page]);
const handleChange = (e) => {
  setFormData({ ...formData, [e.target.name]: e.target.value });
};
const handleVerifyAndProceed = async () => {
```

```
    setValidating(true);
    setError(null);
    // 1. Local Validation
    if (!formData.db_admin_password || formData.db_admin_password.trim() === "") {
      setError("Database Admin Password is required to secure the cluster.");
      setValidating(false);
      return;
    }
    const payload = preparePayload();
    try {
      const response = await fetch('/api/validate', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'x-user-role': userRole // Ensure this is 'viewer' or 'admin'
        },
        body: JSON.stringify(payload)
      });
      const data = await response.json();
      // 2. Handle Errors (STOPS NAVIGATION)
      if (!response.ok) {
        const errorMsg = data.detail || "Validation failed";
        // Handle Existing Cluster (Health Modal)
        if (typeof errorMsg === 'string' && (errorMsg.includes("EXISTS|") ||
errorMsg.includes("Conflict Error"))) {
          const cleanMsg = errorMsg.replace("EXISTS|", "").replace("Conflict Error: ",
"");
          const parsedNodes = parseConflictData(cleanMsg);
          setConflictNodes(parsedNodes);
          setIsConflictModalOpen(true);
          setError(null);
        }

        // Handle Unauthorized Viewer (Auth Modal)
        else if (typeof errorMsg === 'string' &&
errorMsg.includes("AUTHORIZATION_ERROR|")) {
          const cleanMsg = errorMsg.replace("AUTHORIZATION_ERROR|", "");
          setAuthErrorMessage(cleanMsg);
          setIsAuthModalOpen(true);      // Trigger Pop-up
          setError(null);
        }

        else {
          if (Array.isArray(data.detail)) {
            // If it's a FastAPI schema error, show the first specific error
            const firstError = data.detail[0];
            setError(`Field Error: ${firstError.loc[1]} - ${firstError.msg}`);
          } else {
            // If it's a custom error message from validator.py, show it directly
            setError(errorMsg);
```

```
        }
      }

      setValidating(false);
      return; // IMPORTANT: This prevents moving to Page 2
    }
    // 3. SUCCESS LOGIC (Only runs for Authorized Admin)
    // If we are here, response.ok is true
    setLogs([]);
    setReport(null);
    setPage(2); // MOVE TO PAGE 2 ONLY HERE
    handlePreview(payload);
  } catch (err) {
    setError(`System Error: ${err.message}`);
  } finally {
    setValidating(false);
  }
};
const handlePreview = async (payload) => {
  try {
    const response = await fetch('/api/preview-config', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
    const data = await response.json();
    if (response.ok) setPreviewContent(data.content);
  } catch (err) {
    console.error("Preview failed", err);
  }
};
const downloadLogs = () => {
  const element = document.createElement("a");
  const file = new Blob([logs.join("\n")], {type: 'text/plain'});
  element.href = URL.createObjectURL(file);
  element.download = `harbor_master_deployment_${new Date().toISOString()}.log`;
  document.body.appendChild(element);
  element.click();
  document.body.removeChild(element);
};
const preparePayload = () => {
  return {
    ...formData,
    db_admin_password: formData.db_admin_password,
    innodb_autoinc_lock_mode: parseInt(formData.innodb_autoinc_lock_mode),
    binlog_expire_logs_seconds: parseInt(formData.binlog_expire_logs_seconds),
    innodb_buffer_pool_instances: parseInt(formData.innodb_buffer_pool_instances),
    wsrep_slave_threads: parseInt(formData.wsrep_slave_threads),
    innodb_undo_tablespaces: parseInt(formData.innodb_undo_tablespaces),
    wsrep_gtid_domain_id: parseInt(formData.wsrep_gtid_domain_id),
```

```
      galera_nodes: [
        { ip: formData.node1_ip, node_name: formData.node1_name, server_id: 0 },
        { ip: formData.node2_ip, node_name: formData.node2_name, server_id: 0 },
        { ip: formData.node3_ip, node_name: formData.node3_name, server_id: 0 }
      ],
      lvs_ips: [formData.lvs1_ip, formData.lvs2_ip]
    };
  };
  const handleDbProvision = async () => {
    setDbLoading(true);
    setDbError(null);
    // Prepare the payload for the backend
    const payload = {
        db_name: dbForm.name,
        db_user: dbForm.user,
        db_password: dbForm.pass,
        role: dbForm.role,
        ttl: parseInt(dbForm.ttl) || 30,
        vip: formData.lvs_vip, // Assuming your cluster VIP is in formData
        admin_password: formData.db_admin_password // Assuming root pass is in formData
    };
    try {
        const response = await fetch('/api/api/create-db', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        const data = await response.json();
        if (!response.ok) {
            // This is where Case A and Case B are caught
            throw new Error(data.detail || "Provisioning failed");
        }
        alert(`Success: ${data.detail}`);
        // Optional: Clear form on success
        setDbForm({ name: '', user: '', pass: '', role: 'Read-Only' });
    } catch (err) {
        setDbError(err.message);
    } finally {
        setDbLoading(false);
    }
};
  const executeDeployment = async () => {
    setLoading(true);
    setError(null);
    setLogs([]);

    // Reset progress and milestones to original state
    setProgress(0);
    setMilestones({
        mariadb: "pending",
```

```
        galera: "pending",
        lvs: "pending",
        async: "pending",
        monitoring: "pending"
    });
    const payload = preparePayload();
    try {
      const response = await fetch('/api/deploy', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', 'x-user-role': userRole },
        body: JSON.stringify(payload)
      });
      const data = await response.json();
      if (!response.ok) {
        if (typeof data.detail === 'string' &&
data.detail.includes("AUTHORIZATION_ERROR")) {
          setAuthErrorMessage(data.detail.replace("AUTHORIZATION_ERROR|", ""));
          setIsAuthModalOpen(true);
          return; // Stop the function here
        }
        throw new Error(data.detail || "Deployment failed");
      }
           setReport(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };
  const RequiredLabel = ({ children }) => (
    <label style={miniLabel}>{children} <span style={{ color: '#E91E63'
}}>*</span></label>
  );
  const getGrafanaURL = () => `http://${formData.monitor_ip}:3000/dashboards`;
  const handleCreateDB = async () => {
    if (!dbForm.name || !dbForm.user || !dbForm.pass|| !dbForm.ttl) {
        setDbMessage({ type: 'error', text: "All fields are required!" });
        return;
    }
    setDbLoading(true);
    setDbMessage(null);
    try {
        const response = await fetch('/api/api/create-db', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                db_name: dbForm.name,
                db_user: dbForm.user,
                db_password: dbForm.pass,
               ttl: parseInt(dbForm.ttl),
```

```
                admin_password: formData.db_admin_password, // From Page 1
                vip: formData.lvs_vip // From Page 1
            })
        });
        const data = await response.json();
        if (response.ok) {
            setDbMessage({ type: 'success', text: data.message });
            setDbForm({ name: '', user: '', pass: '',ttl: 30 }); // Reset
        } else {
            throw new Error(data.detail);
        }
    } catch (err) {
        setDbMessage({ type: 'error', text: err.message });
    } finally {
        setDbLoading(false);
    }
};
  // --- LOGIN PAGE RENDER ---
  if (!isAuthenticated) {
    return (
      <div style={{ display: 'flex', justifyContent: 'center', alignItems: 'center',
minHeight: '100vh', backgroundColor: '#f5f2f9', fontFamily: 'Inter, sans-serif' }}>
        <div style={{ ...formCard, width: '400px', padding: '40px', textAlign: 'center',
boxShadow: '0 10px 25px rgba(95, 37, 159, 0.1)' }}>
          <div style={{ backgroundColor: '#5f259f', width: '60px', height: '60px',
borderRadius: '15px', display: 'flex', justifyContent: 'center', alignItems: 'center',
margin: '0 auto 20px' }}>
            <Server color="white" size={32} />
          </div>
          <h2 style={{ color: '#5f259f', marginBottom: '10px', fontSize: '24px' }}>Harbor
Master</h2>
          <p style={{ color: '#666', fontSize: '14px', marginBottom: '30px' }}>Management
Control Plane Login</p>

          <form onSubmit={handleLoginSubmit} style={{ textAlign: 'left' }}>
            <div style={{ marginBottom: '20px' }}>
              <RequiredLabel>Username</RequiredLabel>
              <div style={passwordWrapper}>
                 <User size={18} style={{ position: 'absolute', left: '12px', color:
'#7e57c2' }} />
                  <input name="username" style={{ ...inputStyle, paddingLeft: '40px',
marginBottom: 0 }} placeholder="Enter username" value={loginData.username}
onChange={handleLoginChange} required />
              </div>
            </div>
            <div style={{ marginBottom: '25px' }}>
              <RequiredLabel>Password</RequiredLabel>
              <div style={passwordWrapper}>
                <Lock size={18} style={{ position: 'absolute', left: '12px', color:
'#7e57c2' }} />
```

```
                <input type="password" name="password" style={{ ...inputStyle,
paddingLeft: '40px', marginBottom: 0 }} placeholder="Enter password"
value={loginData.password} onChange={handleLoginChange} required />
              </div>
            </div>
            {loginError && <div style={{ color: '#E91E63', fontSize: '13px', marginBottom:
'15px', textAlign: 'center' }}>{loginError}</div>}
            <button type="submit" style={deployBtn}>
              <LogIn size={20} /> ACCESS DASHBOARD
            </button>
          </form>
          <div style={{ marginTop: '30px', fontSize: '11px', color: '#999' }}>
            Secure Infrastructure Gateway v2.0
          </div>
        </div>
      </div>
    );
  }
  // --- MAIN APP RENDER ---
  return (
    <div style={{ padding: '20px', fontFamily: 'Inter, sans-serif', maxWidth: '1600px',
margin: '0 auto', backgroundColor: '#f5f2f9', minHeight: '100vh' }}>
      <style>{`
        @keyframes spin { from { transform: rotate(0deg); } to { transform:
rotate(360deg); } }
        .animate-spin-loop { animation: spin 1s linear infinite; }
      `}</style>

      <div style={{ backgroundColor: '#5f259f', padding: '24px', borderRadius: '16px',
marginBottom: '20px', color: 'white', display: 'flex', justifyContent: 'space-between',
alignItems: 'center', boxShadow: '0 4px 12px rgba(95, 37, 159, 0.2)' }}>
        <div style={{ display: 'flex', alignItems: 'center', gap: '15px' }}>
            <h1 style={{ margin: 0, fontSize: '26px', letterSpacing: '-0.5px' }}> Harbor
Master</h1>
            <div style={{ background: 'rgba(255,255,255,0.2)', padding: '4px 12px',
borderRadius: '20px', fontSize: '12px' }}>
                <User size={12} style={{ display: 'inline', marginRight: '5px' }} />
{loginData.username}
            </div>
        </div>
        <div style={{ display: 'flex', gap: '20px', alignItems: 'center' }}>
            <div style={{ width: '320px' }}>
            <label style={{ fontSize: '11px', fontWeight: '700', display: 'block',
marginBottom: '6px', color: '#ede7f6' }}>MARIADB ENGINE VERSION *</label>
            <select name="mariadb_version" value={formData.mariadb_version}
onChange={handleChange} style={headerSelect} disabled={loading || validating}>
                <option value="10.6.16">MariaDB 10.6.16</option>
                <option value="10.6.21">MariaDB 10.6.21</option>
                <option value="10.11.16">MariaDB 10.11.16</option>
            </select>
```

```
            </div>
            <button onClick={() => setIsAuthenticated(false)} style={{ background: 'none',
border: '1px solid rgba(255,255,255,0.3)', color: 'white', padding: '8px 15px',
borderRadius: '8px', cursor: 'pointer', fontSize: '12px' }}>Logout</button>
          </div>
        </div>
        {page === 1 ? (
          <div style={{ display: 'flex', flexDirection: 'column', gap: '20px' }}>
            {error && (
              <div style={{ ...errorBox, display: 'flex', alignItems: 'center', gap: '10px',
marginBottom: '10px' }}>
                <AlertTriangle size={20} />
                <span><strong>Configuration Error:</strong> {error}</span>
              </div>
            )}
            <div style={{ display: 'grid', gridTemplateColumns: '1fr 360px', gap: '20px' }}>
              <div style={formCard}>
                <h3 style={cardTitle}><Database size={18} color="#6739B7" /> Cluster
Infrastructure</h3>
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '12px'
}}>
                  {[1, 2, 3].map(i => (
                    <React.Fragment key={i}>
                      <div><RequiredLabel>galera-node-{i} (IP)</RequiredLabel><input
name={`node${i}_ip`} value={formData[`node${i}_ip`]} onChange={handleChange}
style={inputStyle} /></div>
                      <div><RequiredLabel>node-name-{i}</RequiredLabel><input
name={`node${i}_name`} value={formData[`node${i}_name`]} onChange={handleChange}
style={inputStyle} /></div>
                    </React.Fragment>
                  ))}
                </div>
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '12px',
marginTop: '15px', borderTop: '1px solid #f3e5f5', paddingTop: '15px' }}>
                  <div><RequiredLabel>lvs-1 IP</RequiredLabel><input name="lvs1_ip"
value={formData.lvs1_ip} onChange={handleChange} style={inputStyle} /></div>
                  <div><RequiredLabel>lvs-2 IP</RequiredLabel><input name="lvs2_ip"
value={formData.lvs2_ip} onChange={handleChange} style={inputStyle} /></div>
                  <div><RequiredLabel>async-node IP</RequiredLabel><input name="async_ip"
value={formData.async_ip} onChange={handleChange} style={inputStyle} /></div>
                  <div><RequiredLabel>monitor-node IP</RequiredLabel><input
name="monitor_ip" value={formData.monitor_ip} onChange={handleChange} style={inputStyle}
/></div>
                  <div style={{ gridColumn: 'span 2' }}><RequiredLabel>virtual-ip
(VIP)</RequiredLabel><input name="lvs_vip" value={formData.lvs_vip}
onChange={handleChange} style={inputStyle} /></div>
                </div>
              </div>
              <div style={{ display: 'flex', flexDirection: 'column', gap: '20px' }}>
                <div style={formCard}>
```

```
                  <h3 style={cardTitle}><Lock size={18} color="#6739B7" /> SST
Security</h3>
                  <RequiredLabel>wsrep_sst_method</RequiredLabel><input
name="wsrep_sst_method" value={formData.wsrep_sst_method} onChange={handleChange}
style={inputStyle} />
                  <RequiredLabel>wsrep_sst_auth (user:pass)</RequiredLabel>
                  <div style={passwordWrapper}>
                    <input type={showSSTPass ? "text" : "password"} name="wsrep_sst_auth"
value={formData.wsrep_sst_auth} onChange={handleChange} style={inputStylePassword} />
                    <button type="button" onClick={() => setShowSSTPass(!showSSTPass)}
style={eyeBtn}>{showSSTPass ? <EyeOff size={16} /> : <Eye size={16} />}</button>
                  </div>
                </div>
                <div style={formCard}>
  <h3 style={cardTitle}><Lock size={18} color="#6739B7" /> DB Admin Security</h3>
  <RequiredLabel>db_admin_password</RequiredLabel>
  <div style={passwordWrapper}>
    <input
      type={showDBAdminPass ? "text" : "password"}
      name="db_admin_password"
      value={formData.db_admin_password}
      onChange={handleChange}
      style={inputStylePassword}
      placeholder="Enter MariaDB Root Password"
    />
    <button type="button" onClick={() => setShowDBAdminPass(!showDBAdminPass)}
style={eyeBtn}>
      {showDBAdminPass ? <EyeOff size={16} /> : <Eye size={16} />}
    </button>
  </div>
</div>
                <div style={formCard}>
                  <h3 style={cardTitle}><ShieldCheck size={18} color="#6739B7" />
Replication</h3>
                  <RequiredLabel>repl_user</RequiredLabel><input name="repl_user"
value={formData.repl_user} onChange={handleChange} style={inputStyle} />
                  <RequiredLabel>repl_password</RequiredLabel>
                  <div style={passwordWrapper}>
                    <input type={showReplPass ? "text" : "password"} name="repl_password"
value={formData.repl_password} onChange={handleChange} style={inputStylePassword} />
                    <button type="button" onClick={() => setShowReplPass(!showReplPass)}
style={eyeBtn}>{showReplPass ? <EyeOff size={16} /> : <Eye size={16} />}</button>
                  </div>
                </div>
              <div style={formCard}>
  <h3 style={cardTitle}><Network size={18} color="#6739B7" /> Cluster Identity</h3>
  <div style={{ display: 'flex', alignItems: 'center' }}>
    <RequiredLabel>wsrep_cluster_name</RequiredLabel>
    {/* TOOLTIP START */}
    <div
```

```
      style={{ position: 'relative', display: 'inline-flex', marginLeft: '8px' }}
      onMouseEnter={() => setActiveTooltip('clusterName')}
      onMouseLeave={() => setActiveTooltip(null)}
    >
      <HelpCircle size={14} color="#999" />
      <span style={{
        ...tooltipText,
        ...(activeTooltip === 'clusterName' ? tooltipVisible : {})
      }}>
        A unique name for your cluster. All nodes must have the exact same name to
connect.
      </span>
    </div>
    {/* TOOLTIP END */}
  </div>
  <input name="wsrep_cluster_name" value={formData.wsrep_cluster_name}
onChange={handleChange} style={inputStyle} />
  <div style={{ display: 'flex', alignItems: 'center', marginTop: '10px' }}>
    <RequiredLabel>wsrep_provider</RequiredLabel>
    {/* TOOLTIP START */}
    <div
      style={{ position: 'relative', display: 'inline-flex', marginLeft: '8px' }}
      onMouseEnter={() => setActiveTooltip('provider')}
      onMouseLeave={() => setActiveTooltip(null)}
    >
      <HelpCircle size={14} color="#999" />
      <span style={{
        ...tooltipText,
        ...(activeTooltip === 'provider' ? tooltipVisible : {})
      }}>
        The path to the Galera library. <br/>
        <b>Debian/Ubuntu:</b> /usr/lib/galera/... <br/>
        <b>RedHat/CentOS:</b> /usr/lib64/galera/...
      </span>
    </div>
    {/* TOOLTIP END */}
  </div>
  <select name="wsrep_provider" value={formData.wsrep_provider} onChange={handleChange}
style={inputStyle}>
    <option
value="/usr/lib/galera/libgalera_smm.so">/usr/lib/galera/libgalera_smm.so</option>
    <option
value="/usr/lib64/galera/libgalera_smm.so">/usr/lib64/galera/libgalera_smm.so</option>
  </select>
</div>
            </div>
          </div>
          <div style={{ ...formCard, borderTop: '6px solid #6739B7' }}>
    <h3 style={{ ...cardTitle, color: '#6739B7' }}>
        <Layers size={18} /> Version Specific Parameters: {formData.mariadb_version}
```

```
    </h3>
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '30px' }}>

        {/* Modern Parameters Section (Always shown for 10.6 and 10.11) */}
        <div>
            <label style={miniLabel}>wsrep_mode</label>
            <textarea
                name="wsrep_mode"
                value={formData.wsrep_mode}
                onChange={handleChange}
                style={{ ...inputStyle, height: '110px', resize: 'none' }}
                placeholder="e.g., REQUIRED_PRIMARY_KEY,STRICT_REPLICATION"
            />
        </div>
        <div style={{ display: 'flex', flexDirection: 'column', gap: '10px' }}>
            <div>
                <label style={miniLabel}>binlog_expire_logs_seconds</label>
                <input
                    name="binlog_expire_logs_seconds"
                    value={formData.binlog_expire_logs_seconds}
                    onChange={handleChange}
                    style={inputStyle}
                />
                <span style={{fontSize: '10px', color: '#7e57c2'}}>7 days = 604800s</span>
            </div>
            <div>
                <label style={miniLabel}>innodb_buffer_pool_instances</label>
                <input
                    name="innodb_buffer_pool_instances"
                    value={formData.innodb_buffer_pool_instances}
                    onChange={handleChange}
                    style={inputStyle}
                />
            </div>
        </div>
        {/* Extended Parameters (Shown only for 10.11.16) */}
        {formData.mariadb_version === "10.11.16" && (
            <div style={{
                gridColumn: 'span 2',
                display: 'grid',
                gridTemplateColumns: '1fr 1fr 1fr',
                gap: '15px',
                marginTop: '10px',
                paddingTop: '10px',
                borderTop: '1px dashed #d1c4e9'
            }}>
                <div>
                    <label style={miniLabel}>wsrep_slave_threads</label>
                    <input name="wsrep_slave_threads" value={formData.wsrep_slave_threads}
onChange={handleChange} style={inputStyle} />
```

```
                </div>
                <div>
                    <label style={miniLabel}>innodb_undo_tablespaces</label>
                    <input name="innodb_undo_tablespaces"
value={formData.innodb_undo_tablespaces} onChange={handleChange} style={inputStyle} />
                </div>
                <div>
                    <label style={miniLabel}>wsrep_gtid_domain_id</label>
                    <input name="wsrep_gtid_domain_id"
value={formData.wsrep_gtid_domain_id} onChange={handleChange} style={inputStyle} />
                </div>
            </div>
        )}
    </div>
</div>
          <div style={{ display: 'flex', justifyContent: 'flex-end', gap: '15px' }}>
            <button onClick={handleVerifyAndProceed} disabled={validating} style={{
...deployBtn, width: '350px', backgroundColor: validating ? '#9575cd' : '#6739B7' }}>
              {validating ? <Loader2 className="animate-spin-loop" size={20} /> :
<ShieldCheck size={20} />}
              {validating ? "VERIFYING PARAMETERS..." : "VERIFY & PROCEED"}
            </button>
          </div>
        </div>
      ) : (
        <div style={{ display: 'grid', gridTemplateColumns: '360px 1fr', gap: '20px',
alignItems: 'start' }}>

          {/* LEFT COLUMN: Health Reports */}
          <div style={{ display: 'flex', flexDirection: 'column', gap: '20px' }}>
              <button
                onClick={() => {
  setPage(1);
  setError(null);
  setShowPreview(false);
  // Reset Deployment States
  setProgress(0);
  setMilestones({
    mariadb: "pending",
    galera: "pending",
    lvs: "pending",
    async: "pending",
    monitoring: "pending"
  });
  setLogs([]); // Optional: Clear logs too for a fresh start
}}
                disabled={loading}
                style={{ ...deployBtn, backgroundColor: loading ? '#b39ddb' : '#7e57c2',
padding: '10px', cursor: loading ? 'not-allowed' : 'pointer', opacity: loading ? 0.7 : 1
}}
```

```
              >
                <ArrowLeft size={18} /> BACK TO CONFIG
              </button>
            {report && (
              <div style={reportContainer}>
                <div style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'center', marginBottom: '15px' }}>
                    <h2 style={{ color: '#5f259f', margin: 0, fontSize: '18px'
}}>{report.status}</h2>
                    <div style={{
                        backgroundColor: report.health_report.overall_status === "Healthy"
? "#166534" :
                                         report.health_report.overall_status === "CRITICAL"
? "#b91c1c" : "#f59e0b",
                        color: 'white', padding: '4px 10px', borderRadius: '20px',
fontSize: '11px', fontWeight: 'bold'
                    }}>
                        {report.health_report.overall_status}
                    </div>
                </div>
                <a href={getGrafanaURL()} target="_blank" rel="noopener noreferrer"
style={grafanaBtn}>
                    <Activity size={18} /> OPEN MONITORING DASHBOARD
                </a>
                {/* GALERA SECTION */}
                <div style={healthSection}>
                    <div style={sectionLabel}>
                        <Database size={14}/> Galera Cluster: {report.cluster_name}
                        <span style={{ marginLeft: 'auto', color: '#6739B7' }}>Size:
{report.health_report.cluster_size}</span>
                    </div>
                    {report.health_report.galera.map((node, idx) => (
                        <div key={idx} style={healthRow}>
                            <span style={{ fontSize: '12px' }}>{node.host}</span>
                            <span style={{ color: node.sync_state === "Synced" ? "#166534"
: "#b91c1c", fontSize: '11px', fontWeight: 'bold' }}>
                                {node.sync_state}
                            </span>
                        </div>
                    ))}
                </div>
                {/* ASYNC SECTION */}
                <div style={healthSection}>
                    <div style={sectionLabel}><Globe size={14}/> Async Replica</div>
                    {report.health_report.async ? (
                        <div style={healthRow}>
                            <span style={{ fontSize: '12px'
}}>{report.health_report.async.host}</span>
                            <div style={{ display: 'flex', gap: '5px' }}>
```

```
                                <span style={pill(report.health_report.async.io === "Yes"
|| report.health_report.async.io_running === "Yes")}>IO</span>
                                <span style={pill(report.health_report.async.sql === "Yes"
|| report.health_report.async.sql_running === "Yes")}>SQL</span>
                            </div>
                        </div>
                    ) : <div style={{fontSize: '11px', color: '#999'}}>No Async Node
Configured</div>}
                </div>
                {/* LVS HEALTH SECTION */}
                <div style={healthSection}>
                    <div style={sectionLabel}><Network size={14}/> LVS Load
Balancers</div>
                    <div style={{ fontSize: '10px', marginBottom: '8px', color: '#5f259f',
display: 'flex', justifyContent: 'space-between' }}>
                        <span>VIP: {report.lvs_vip}</span>
                        {report.health_report.active_writer_ip && (
                            <span style={{fontWeight: 'bold'}}>WRITER:
{report.health_report.active_writer_ip}</span>
                        )}
                    </div>
                    {report.health_report.lvs.map((lvs, idx) => (
                        <div key={idx} style={{...healthRow, flexDirection: 'column',
alignItems: 'flex-start', gap: '4px', padding: '8px 0'}}>
                            <div style={{ display: 'flex', justifyContent:
'space-between', width: '100%', alignItems: 'center' }}>
                                <span style={{ fontSize: '12px', fontWeight: '500'
}}>{lvs.host}</span>
                                <div style={{ display: 'flex', gap: '8px', alignItems:
'center' }}>
                                    <span style={{ fontSize: '9px', color: lvs.ssh ||
lvs.ssh_reachable ? '#166534' : '#b91c1c' }}>
                                        {lvs.ssh || lvs.ssh_reachable ? "SSH OK" : "SSH
FAIL"}
                                    </span>
                                    {lvs.vip || lvs.holds_vip ? <Zap size={14}
color="#f8961e" title="Holds VIP" /> : null}
                                    <CheckCircle size={14} color={lvs.target ||
lvs.pointing_to ? "#166534" : "#ccc"} />
                                </div>
                            </div>
                            {(lvs.target || lvs.pointing_to) && (
                                <div style={{ fontSize: '10px', color: '#666', fontStyle:
'italic' }}>
                                    ↳ Routing to: {lvs.target || lvs.pointing_to}
                                </div>
                            )}
                        </div>
                    ))}
                </div>
```

```
              </div>
            )}
{/* 2. PROVISION CUSTOM DATABASE */}
{progress === 100 && (
    <div style={formCard}>
        <h3 style={cardTitle}><Database size={18} color="#6739B7" /> Provision Custom
Database</h3>
        <p style={{ fontSize: '12px', color: '#666', marginBottom: '15px' }}>
            Create an isolated database and user on the new cluster via LVS VIP.
        </p>
        <div style={{ display: 'flex', flexDirection: 'column', gap: '15px' }}>
            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px' }}>
                {/* Database Name with Normalization & Forbidden Check */}
                <div>
                    <RequiredLabel>Database Name</RequiredLabel>
                    <input
                        style={{
                            ...inputStyle,
                            borderColor: ['mysql', 'root', 'admin', 'sys',
'information_schema', 'performance_schema'].includes(dbForm.name) ? '#e53935' : '#e0e0e0'
                        }}
                        placeholder="e.g. app_prod"
                        value={dbForm.name}
                        onChange={(e) => setDbForm({ ...dbForm, name:
e.target.value.toLowerCase().replace(/\s/g, '') })}
                    />
                    {['mysql', 'root', 'admin', 'sys', 'information_schema',
'performance_schema'].includes(dbForm.name) && (
                        <span style={{ color: '#e53935', fontSize: '10px' }}>Reserved
system name forbidden.</span>
                    )}
                </div>
                {/* Username with Normalization */}
                <div>
                    <RequiredLabel>Username</RequiredLabel>
                    <input
                        style={inputStyle}
                        placeholder="e.g. app_user"
                        value={dbForm.user}
                        onChange={(e) => setDbForm({ ...dbForm, user:
e.target.value.toLowerCase().replace(/\s/g, '') })}
                    />
                </div>
            </div>
            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px' }}>
                {/* Password Field */}
                <div>
                    <RequiredLabel>Password</RequiredLabel>
                    <div style={{ position: 'relative', display: 'flex', alignItems:
'center' }}>
```

```
                        <input
                            type={showDbPassword ? "text" : "password"}
                            style={{ ...inputStyle, marginBottom: 0, paddingRight: '45px'
}}
                            placeholder="••••••••"
                            value={dbForm.pass}
                            onChange={(e) => setDbForm({ ...dbForm, pass: e.target.value
})}
                        />
                        <button
                            type="button"
                            onClick={() => setShowDbPassword(!showDbPassword)}
                            style={{ position: 'absolute', right: '10px', background:
'none', border: 'none', cursor: 'pointer', color: '#6739B7' }}
                        >
                            {showDbPassword ? <EyeOff size={18} /> : <Eye size={18} />}
                        </button>
                    </div>
                </div>
                {/* Role Dropdown */}
                <div>
                    <RequiredLabel>Privilege Role</RequiredLabel>
                    <select
                        style={{ ...inputStyle, backgroundColor: '#fff', cursor: 'pointer'
}}
                        value={dbForm.role}
                        onChange={(e) => setDbForm({ ...dbForm, role: e.target.value })}
                    >
                        <option value="Read-Only">Read-Only (SELECT, SHOW VIEW)</option>
                        <option value="Read-Write">Read-Write (SELECT, INSERT, UPDATE,
DELETE)</option>
                        <option value="DDL-Admin">Owner/Admin (ALL PRIVILEGES)</option>
                    </select>
                </div>
            </div>
            <div>
        <RequiredLabel>Password TTL (Days)</RequiredLabel>
        <input
            type="number"
            style={inputStyle}
            placeholder="e.g. 30"
            min="1"
            max="365"
            value={dbForm.ttl}
            onChange={(e) => setDbForm({ ...dbForm, ttl: e.target.value })}
        />
        <p style={{ fontSize: '10px', color: '#666', marginTop: '2px' }}>
            User password expires after {dbForm.ttl || 30} days.
        </p>
    </div>
```

```
            {/* SRE Conflict Detection Messaging */}
            {/* Note: In a real app, these triggers would come from your backend check API
*/}
            {dbError && (
                <div style={{ backgroundColor: '#ffebee', padding: '10px', borderRadius:
'4px', border: '1px solid #ef9a9a', display: 'flex', alignItems: 'center', gap: '8px' }}>
                    <AlertTriangle size={16} color="#c62828" />
                    <span style={{ fontSize: '12px', color: '#c62828' }}>{dbError}</span>
                </div>
            )}
            <button
                onClick={handleDbProvision}
                disabled={loading || !dbForm.name || !dbForm.user || !dbForm.pass}
                style={{ ...deployBtn, backgroundColor: '#6739B7', marginTop: '5px' }}
            >
                <ShieldCheck size={18} /> PROVISION DATABASE
            </button>
        </div>
    </div>
)}
    {/* 3. PLACEHOLDER (Existing) */}
    {!report && (
        <div style={emptyResults}>
            <div style={{ opacity: 0.5 }}>{loading ? <Loader2
className="animate-spin-loop" size={40} /> : <Cpu size={40} />}</div>
            <div style={{ marginTop: '10px' }}>{loading ? "Orchestrating MariaDB HA..." :
"Ready for Deployment"}</div>
        </div>
    )}
    {/* 4. ERROR BOX (Existing) */}
    {error && (
        <div style={{ ...errorBox, marginTop: '10px' }}>
            <div style={{ display: 'flex', gap: '10px', alignItems: 'center' }}>
                <AlertTriangle size={18} />
                <span><strong>Deployment Error:</strong> {error}</span>
            </div>
        </div>
    )}
</div>
        {/* RIGHT COLUMN: Progress, Actions, and Console */}
          <div style={{ display: 'flex', flexDirection: 'column', height: 'calc(100vh -
100px)', gap: '15px' }}>
            {/* ENHANCED PROGRESS UI WITH ERROR HANDLING */}
            <div style={formCard}>
                <div style={{ display: 'flex', justifyContent: 'space-between',
marginBottom: '8px' }}>
                    <span style={{ fontSize: '12px', fontWeight: 'bold', color: '#5f259f'
}}>Deployment Progress</span>
                    <span style={{ fontSize: '12px', fontWeight: 'bold', color: '#5f259f'
}}>{progress}%</span>
```

```
                </div>
                {/* Progress Bar - Turns RED if any milestone failed */}
                <div style={{ width: '100%', height: '8px', backgroundColor: '#e0e0e0',
borderRadius: '4px', overflow: 'hidden' }}>
                    <div style={{
                        width: `${progress}%`,
                        height: '100%',
                        backgroundColor: Object.values(milestones).includes("failed") ?
"#e53935" : "#6739B7",
                        transition: 'width 0.5s ease-in-out'
                    }} />
                </div>
                <div style={{ display: 'grid', gridTemplateColumns: 'repeat(5, 1fr)', gap:
'10px', marginTop: '20px' }}>
                    {Object.entries(milestones).map(([key, status]) => (
                        <div key={key} style={{
                            padding: '10px', borderRadius: '8px', textAlign: 'center',
fontSize: '11px', fontWeight: 'bold',
                            backgroundColor:
                                status === 'done' ? '#e8f5e9' :
                                status === 'failed' ? '#ffebee' :
                                status === 'loading' ? '#fff3e0' : '#f5f5f5',
                            color:
                                status === 'done' ? '#2e7d32' :
                                status === 'failed' ? '#c62828' :
                                status === 'loading' ? '#ef6c00' : '#9e9e9e',
                            border: `1px solid ${
                                status === 'done' ? '#4caf50' :
                                status === 'failed' ? '#e53935' :
                                status === 'loading' ? '#ff9800' : '#e0e0e0'
                            }`,
                            textTransform: 'uppercase'
                        }}>
                            {status === 'loading' && <Loader2 size={12}
className="animate-spin-loop" style={{marginRight: '5px', display: 'inline'}} />}
                            {status === 'done' && <CheckCircle size={12}
style={{marginRight: '5px', display: 'inline'}} />}
                            {status === 'failed' && <XCircle size={12}
style={{marginRight: '5px', display: 'inline'}} />}
                            {key.replace('_', ' ')}
                        </div>
                    ))}
                </div>
                <p style={{ fontSize: '13px', color: '#666', marginTop: '15px', fontStyle:
'italic', textAlign: 'center' }}>
                    {Object.values(milestones).includes("failed") ? (
                        <span style={{ color: '#e53935', fontWeight: 'bold', display:
'flex', alignItems: 'center', justifyContent: 'center', gap: '8px' }}>
                            <AlertTriangle size={16} /> Deployment halted due to an error.
Check logs below.
```

```
                        </span>
                    ) : (
                        <>
                            {progress < 15 && "Initializing infrastructure handshake..."}
                            {progress >= 15 && progress < 40 && "Installing MariaDB
dependencies and repositories..."}
                            {progress >= 40 && progress < 65 && "Building Galera Cluster
(this may take a few minutes)..."}
                            {progress >= 65 && progress < 80 && "Configuring LVS-DR and
Keepalived high availability..."}
                            {progress >= 80 && progress < 90 && "Setting up Asynchronous
replication and GTID tracking..."}
                            {progress >= 90 && progress < 100 && "Deploying Prometheus
exporters and Grafana dashboards..."}
                            {progress === 100 && "Deployment successful! All systems are
operational."}
                        </>
                    )}
                </p>
            </div>
            {/* ACTION BUTTONS */}
            <div style={{ display: 'flex', gap: '10px' }}>
              <button
                onClick={() => setShowPreview(!showPreview)}
                disabled={loading}
                style={{ ...deployBtn, backgroundColor: '#607d8b', flex: 1 }}
              >
                <FileCode size={20} /> {showPreview ? "HIDE PREVIEW" : "PREVIEW CONFIG"}
              </button>
              {!report && (
                <button
                  onClick={executeDeployment}
                  disabled={loading}
                  style={{ ...deployBtn, backgroundColor: loading ? '#7cb342' : '#4CAF50',
cursor: loading ? 'not-allowed' : 'pointer', flex: 2, boxShadow: '0 4px 10px rgba(76, 175,
80, 0.3)' }}
                >
                  {loading ? <Loader2 size={20} className="animate-spin-loop" /> : <Play
size={20} />}
                  {loading ? "DEPLOYMENT IN PROGRESS..." : "START ORCHESTRATION"}
                </button>
              )}
              {(report || error) && !loading && (
                <button
                  onClick={downloadLogs}
                  style={{ ...deployBtn, backgroundColor: '#2196F3', flex: 1, boxShadow:
'0 4px 10px rgba(33, 150, 243, 0.3)' }}
                >
                  <Download size={20} /> DOWNLOAD LOGS
                </button>
```

```
              )}
            </div>
            {/* CONFIG PREVIEW */}
            {showPreview && (
              <div style={{ ...formCard, backgroundColor: '#1e1e1e', border: '1px solid
#333', padding: '0', overflow: 'hidden' }}>
                <div style={{ background: '#333', padding: '8px 16px', display: 'flex',
justifyContent: 'space-between', alignItems: 'center' }}>
                    <span style={{ color: '#aaa', fontSize: '11px', fontWeight: 'bold',
fontFamily: 'monospace' }}>/etc/mysql/mariadb.conf.d/60-galera.cnf</span>
                </div>
                <pre style={{ margin: 0, padding: '20px', fontSize: '12px', lineHeight:
'1.5', color: '#d4d4d4', background: '#1e1e1e', maxHeight: '300px', overflowY: 'auto',
whiteSpace: 'pre-wrap', fontFamily: '"Fira Code", monospace' }}>
                  {previewContent || "Generating preview..."}
                </pre>
              </div>
            )}
            {/* DEPLOYMENT CONSOLE */}
            <div style={{ ...formCard, backgroundColor: '#0d1117', borderLeft: '4px solid
#6739B7', display: 'flex', flexDirection: 'column', flex: 1,
maxHeight:'none',marginBottom: '0',overflow: 'hidden' }}>
                <div style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'center', marginBottom: '16px' }}>
                  <h3 style={{ ...cardTitle, color: '#d1c4e9', margin: 0 }}><Terminal
size={18} /> Deployment Console</h3>
                  <span style={{ fontSize: '10px', padding: '2px 8px', borderRadius:
'10px', backgroundColor: streamStatus === "Connected" ? "#166534" : "#b91c1c", color:
'white', fontWeight: 'bold' }}>{streamStatus.toUpperCase()}</span>
                </div>
                <div ref={logContainerRef} style={{ ...consoleBox,flex: 1,overflowY:
'auto',height: '100%'}}>
                    {logs.length === 0 && !loading && <div style={{color:
'#4c566a'}}>Terminal ready. Click 'START ORCHESTRATION' to begin.</div>}
                    {logs.map((log, idx) => (
                        <div key={idx} style={{ borderBottom: '1px solid #1a1a1a',
padding: '2px 0', whiteSpace: 'pre-wrap', color: log.includes('TASK [') ? '#58a6ff' :
log.includes('PLAY [') ? '#d29922' : log.includes('failed=') && !log.includes('failed=0')
? '#ff5f56' : '#a3be8c', fontSize: '13px' }}>{log}</div>
                    ))}
                </div>
            </div>
          </div>
        </div>
      )}
     <ConflictModal
        isOpen={isConflictModalOpen}
        data={conflictNodes}
        onClose={() => setIsConflictModalOpen(false)}
      />
```

```
        <AuthModal
       isOpen={isAuthModalOpen}
       message={authErrorMessage}
       onClose={() => setIsAuthModalOpen(false)}
     />
     </div>
  );
}
// STYLES
const tooltipContainer = {
    position: 'relative',
    display: 'inline-flex',
    alignItems: 'center',
    marginLeft: '6px',
    cursor: 'pointer',
    verticalAlign: 'middle'
  };
  const tooltipText = {
    visibility: 'hidden',
    width: '240px',
    backgroundColor: '#333',
    color: '#fff',
    textAlign: 'left',
    borderRadius: '6px',
    padding: '10px',
    position: 'absolute',
    zIndex: 10,
    bottom: '125%',
    left: '50%',
    marginLeft: '-120px',
    opacity: 0,
    transition: 'opacity 0.3s',
    fontSize: '12px',
    lineHeight: '1.4',
    boxShadow: '0px 4px 10px rgba(0,0,0,0.2)'
  };
const tooltipVisible = {
  visibility: 'visible',
  opacity: 1
};
const modalOverlay = {
  position: 'fixed',
  top: 0,
  left: 0,
  right: 0,
  bottom: 0,
  backgroundColor: 'rgba(0, 0, 0, 0.75)',
  display: 'flex',
  justifyContent: 'center',
  alignItems: 'center',
```

```
  zIndex: 2000,
  backdropFilter: 'blur(4px)'
};
const modalContent = {
  backgroundColor: '#ffffff',
  padding: '30px',
  borderRadius: '20px',
  width: '500px',
  boxShadow: '0 25px 50px -12px rgba(0, 0, 0, 0.25)',
  textAlign: 'center',
  border: '1px solid #d1c4e9'
};
const badgeStyle = (status) => ({
  backgroundColor: status.toLowerCase() === 'synced' ? '#dcfce7' : '#fee2e2',
  color: status.toLowerCase() === 'synced' ? '#166534' : '#991b1b',
  padding: '4px 10px',
  borderRadius: '6px',
  fontSize: '11px',
  fontWeight: 'bold',
  textTransform: 'uppercase'
});
const modalBtn = {
  marginTop: '20px',
  padding: '12px 24px',
  backgroundColor: '#6739B7',
  color: 'white',
  border: 'none',
  borderRadius: '10px',
  fontWeight: 'bold',
  cursor: 'pointer',
  width: '100%'
};
const formCard = { background: '#ffffff', padding: '20px', borderRadius: '12px',
boxShadow: '0 2px 8px rgba(0,0,0,0.06)' };
const cardTitle = { margin: '0 0 16px 0', fontSize: '15px', display: 'flex', alignItems:
'center', gap: '10px', color: '#512da8', fontWeight: '700' };
const inputStyle = { width: '100%', padding: '10px', borderRadius: '8px', border: '1.5px
solid #ede7f6', fontSize: '13px', boxSizing: 'border-box', marginBottom: '12px' };
const inputStylePassword = { ...inputStyle, paddingRight: '40px', marginBottom: 0 };
const passwordWrapper = { position: 'relative', display: 'flex', alignItems: 'center',
marginBottom: '12px' };
const eyeBtn = { position: 'absolute', right: '12px', background: 'none', border: 'none',
cursor: 'pointer', color: '#6739B7' };
const miniLabel = { fontSize: '11px', color: '#7e57c2', fontWeight: '700', textTransform:
'uppercase', marginBottom: '5px', display: 'block' };
const headerSelect = { width: '100%', padding: '10px', borderRadius: '10px',
backgroundColor: '#6739B7', border: '1.5px solid #9575cd', fontSize: '14px', fontWeight:
'bold', color: 'white' };
const deployBtn = { padding: '16px', background: '#6739B7', color: 'white', border:
'none', borderRadius: '12px', cursor: 'pointer', fontWeight: 'bold', fontSize: '16px',
```

```
display: 'flex', alignItems: 'center', justifyContent: 'center', gap: '12px', width:
'100%' };
const errorBox = { padding: '15px', background: '#fff5f5', border: '1px solid #feb2b2',
color: '#c53030', borderRadius: '12px', fontSize: '13px' };
const reportContainer = { background: '#f3e5f5', padding: '20px', borderRadius: '16px',
border: '1px solid #d1c4e9' };
const healthSection = { backgroundColor: 'rgba(255,255,255,0.6)', padding: '12px',
borderRadius: '10px', marginBottom: '12px', border: '1px solid #ede7f6' };
const sectionLabel = { fontSize: '11px', fontWeight: 'bold', color: '#512da8',
marginBottom: '8px', display: 'flex', alignItems: 'center', gap: '6px', textTransform:
'uppercase' };
const healthRow = { display: 'flex', justifyContent: 'space-between', alignItems:
'center', padding: '4px 0', borderBottom: '1px solid rgba(0,0,0,0.03)' };
const emptyResults = { height: '300px', display: 'flex', flexDirection: 'column',
alignItems: 'center', justifyContent: 'center', color: '#9575cd', border: '2px dashed
#d1c4e9', borderRadius: '16px', textAlign: 'center', padding: '20px' };
const infoBox = { marginTop: '10px', padding: '12px', background: '#ede7f6', borderRadius:
'10px', fontSize: '11px', color: '#512da8', display: 'flex', gap: '10px', alignItems:
'center' };
const pill = (active) => ({ padding: '2px 6px', borderRadius: '4px', fontSize: '9px',
fontWeight: 'bold', backgroundColor: active ? '#166534' : '#b91c1c', color: 'white' });
const grafanaBtn = { display: 'flex', alignItems: 'center', justifyContent: 'center', gap:
'10px', backgroundColor: '#f8961e', color: 'white', padding: '14px', borderRadius: '10px',
textDecoration: 'none', fontWeight: 'bold', fontSize: '13px', marginBottom: '15px',
boxShadow: '0 4px 6px rgba(0,0,0,0.1)' };
const consoleBox = { flex: 1, overflowY: 'auto', fontSize: '11px', fontFamily: '"Fira
Code", monospace', color: '#a3be8c', backgroundColor: '#000', padding: '12px',
borderRadius: '6px', lineHeight: '1.4' };
export default App;
```

npm start

(venv) ubuntu@head-node:~/harbour-master/frontend$ sudo cat Dockerfile

```
None
# Stage 1: Build the React App
FROM node:20-alpine AS build
WORKDIR /app
# Install dependencies first (better caching)
COPY package*.json ./
RUN npm install
# Copy source and build
COPY . .
RUN npm run build
```

```
# Stage 2: Production environment
FROM nginx:alpine
# Copy the build output to replace the default Nginx content
COPY --from=build /app/build /usr/share/nginx/html
# Expose port 80 for Nginx
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

(venv) **ubuntu@head-node**:**~/harbour-master**$ sudo cat docker-compose.yml

```
None
services:
  # --- Reverse Proxy ---
  nginx:
    image: nginx:alpine
    container_name: harbor-nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - frontend
      - backend
    networks:
      - harbor-network
  # --- Application ---
  frontend:
    build: ./frontend
    container_name: harbor-frontend
    networks:
      - harbor-network
  backend:
    build: ./backend
    container_name: harbor-backend
    volumes:
      - /home/ubuntu/.ssh:/home/ubuntu/.ssh:rw  # <--- Does it have this?
    networks:
      - harbor-network
networks:
  harbor-network:
    driver: bridge
```

(venv) **ubuntu@head-node**:**~/harbour-master/nginx**$ sudo cat nginx.conf

None

```
events {}
http {
    server {
        listen 80;
        server_name harbour.master;
        # Route to Frontend
        location / {
            proxy_pass http://harbor-frontend:80;
            proxy_set_header Host $host;
        }
        # SSE Stream - MUST come before the general /api/ block
        location /api/logs/stream {
            proxy_pass http://harbor-backend:8000/api/logs/stream;
            # 1. Standard Proxy Headers
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header Connection ""; # Important: must be empty for SSE

            # 2. Force No-Buffer (The "Real-Time" Switch)
            proxy_buffering off;
            proxy_cache off;
            proxy_limit_rate 0; # Ensure no speed limiting

            # 3. SSE Specific Headers
            proxy_set_header Cache-Control "no-cache";
            proxy_set_header X-Accel-Buffering "no"; # CRITICAL for Nginx/FastAPI SSE

            # 4. Persistence
            proxy_read_timeout 24h;
            proxy_send_timeout 24h;
            keepalive_timeout 24h;
            chunked_transfer_encoding on;
        }
        # General Route to Backend
        location /api/ {
            # Fixed: ensured name is 'harbor-backend'
            proxy_pass http://harbor-backend:8000/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_connect_timeout 600s;
            proxy_send_timeout 600s;
            proxy_read_timeout 600s; # 10 minutes
        }
    }
}
```

Docker Setup

```
sudo apt install -y docker.io docker-compose-plugin
sudo apt-get install -y ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings

 sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update

sudo apt-get install -y docker-compose-plugin

(venv) ubuntu@head-node:~/harbour-master$ docker compose version
Docker Compose version v5.1.0

sudo usermod -aG docker $USER

(venv) ubuntu@head-node:~/harbour-master$ groups
```

(venv) **ubuntu@head-node**:**~/harbour-master**$ docker system prune -a --volumes -f

(venv) **ubuntu@head-node**:**~/harbour-master**$ docker compose build frontend

(venv) **ubuntu@head-node**:**~/harbour-master**$ docker compose build backend

(venv) **ubuntu@head-node**:**~/harbour-master**$ docker compose ps

(venv) **ubuntu@head-node**:**~/harbour-master**$ docker compose logs -f nginx

(venv) **ubuntu@head-node**:**~/harbour-master**$ docker compose logs -f backend
(venv) **ubuntu@head-node**:**~/harbour-master**$ docker compose logs -f frontend

The Workflow:

1. **User** types `harbour.master` in their browser.
2. The request hits the **Nginx** container.
3. **Nginx** looks at `nginx.conf`, sees the `/` route, and fetches the UI from the **Frontend** container.
4. User fills out the form (3 nodes, MariaDB 10.11.16) and hits **Submit**.
5. The browser sends a POST request to `harbour.master/api/deploy`.
6. **Nginx** sees the `/api/` prefix and forwards it to the **Backend** container.
7. The **Backend** (built via `backend/Dockerfile`) triggers Ansible.
8. **Ansible** logs are sent back through the **SSE route** defined in `nginx.conf` so the user sees them live.

SSE VS Websockets

Feature	SSE (Server-Sent Events)	WebSockets
Communication	**Mono-directional** (Server $\rightarrow$ Client)	**Bi-directional** (Full Duplex)
Protocol	Standard HTTP	Upgrades from HTTP to ws://
Complexity	Low (Automatic reconnection)	High (Requires custom heartbeat/logic)
Proxy Friendly	Works with standard Nginx/Firewalls	Often blocked by strict Firewalls/Proxies

---

Why we used SSE in our Harbor-Master Setup

We chose SSE for three specific technical reasons:

- **The Nature of Ansible Logs:** When you run a playbook, the communication is one-way. The server is "shouting" logs at the browser. The browser doesn't need to talk back to the server during the deployment. SSE is designed perfectly for this "Streaming" behavior.
- **Simplicity & Reliability:** SSE runs over standard **HTTP**. It handles dropped connections automatically (it will try to reconnect if the Wi-Fi blips). WebSockets require complex "Handshaking" and custom code to handle disconnections.

- **Nginx Compatibility:** As we saw in our `nginx.conf`, SSE is easy to tune using standard proxy headers. Supporting WebSockets in Nginx requires a more complex configuration for "Protocol Upgrades."

NGINX Role: It acts as a **reverse proxy**. It is the first point of contact inside the datacenter, responsible for:

- Terminating the connection (SSL/TLS).
- Applying security rules

SSL (Secure Sockets Layer) and its modern successor, **TLS (Transport Layer Security)**, are cryptographic protocols that establish an **encrypted, authenticated, and integrity-checked connection** between a client (like a browser) and a server. Their purpose is to secure data transmitted over a network, preventing eavesdropping and tampering. This security is achieved through a **digital certificate** (known as an SSL/TLS certificate) used during a "handshake" process to verify the server's identity and agree on a secure session key for data encryption. When you see a website starting with `https://`, it signifies that an active TLS connection is securing your communication

NGINX acts as a powerful **gateway** and **traffic manager** for the backend servers:

- **Reverse Proxy:** Hides the application servers, acting as a security barrier and routing requests.
- **SSL Termination:** Decrypts incoming secure traffic from the CDN, relieving the application servers of the processing load.
- **Load Balancing:** Distributes traffic evenly across multiple application containers for high availability and performance (using methods like Round Robin).
- **Security Filter:** Blocks all requests that do not originate from the trusted CDN (Cloudflare/Akamai) IP ranges, ensuring only validated traffic enters the core application environment.

1. The Request Entry: Port 80 (The Front Door)

When a user types your Head Node's IP address into a browser, the request hits the **Physical Network Interface** of your VM.

- **Port involved: 80** (HTTP) or **443** (HTTPS).
- **Docker Concept (Port Mapping):** In your `docker-compose.yml`, you likely have `ports: - "80:80"`. This tells the Host OS: "Any traffic hitting the real VM on port 80 should be tunneled directly into the Nginx container."

2. The Router: Nginx (Reverse Proxy)

Nginx receives the request and looks at the URL path to decide where to send it. This is the **Reverse Proxy** concept.

- **Scenario A (Frontend):** If the user asks for / (the home page), Nginx serves the React/Vue static files.
- **Scenario B (Backend):** If the user asks for /api/deploy, Nginx sees the /api prefix and forwards the request.

3. The Internal Handshake: Docker Bridge Networking

This is where the "Magic" happens. Nginx needs to talk to the Backend container.

- **Docker Concept (Service Discovery):** Docker creates a private virtual network (the **Bridge Network**). Inside this network, containers can talk to each other using their **Service Names** as hostnames.
- **Workflow:** Nginx doesn't need to know the Backend's IP. It just sends the request to http://backend:8000. Docker's internal DNS resolves "backend" to the correct container.
- **Port involved: 8000** (Internal only). Notice that port 8000 does **not** need to be open on your physical VM firewall; it only exists inside the Docker network for Nginx to see.

4. The Processing: Backend (Uvicorn/FastAPI)

The Backend container receives the request on port 8000.

- **Example:** Let's say the request is POST /api/deploy-cluster.
- **Workflow:** The Python code starts executing. It looks at the parameters (how many Galera nodes, which IPs).
- **Docker Concept (Isolation):** The Python code thinks it is running on its own computer. It has its own venv, its own temporary files, and its own process ID (PID 1).

5. The "Reach Out": From Container to Bare Metal

Now the Backend needs to actually install MariaDB on your remote Galera nodes.

- **The Problem:** The Backend is inside a Docker "bubble." How does it talk to the real world?
- **The Solution:** It uses the **Docker Gateway**. It reaches out through the virtual network interface to the Host VM, and then via SSH (Port 22) to the Galera/LVS nodes.
- **Workflow:** Python (inside container) → Docker Bridge → Host Eth0 → Remote Galera Node (SSH).

How Control Node is talking to the db nodes?

1. The Key Pair (The "Lock and Key")

The process starts on the Head Node, where an SSH Key Pair is generated (usually using `ssh-keygen`).

- **Private Key (`id_rsa`):** Stays strictly on the Head Node. Think of this as the physical key in your pocket.
- **Public Key (`id_rsa.pub`):** This is copied to the DB Nodes. Think of this as the "lock" installed on the door.

2. The Authentication Flow

When you run an Ansible playbook, the following "Handshake" happens:

1. **Request:** The Head Node initiates a connection to the DB Node's IP on **Port 22**.
2. **Challenge:** The DB Node sends a "challenge" message back.
3. **Sign:** The Head Node signs that challenge using its **Private Key** and sends it back.
4. **Verify:** The DB Node uses the **Public Key** stored in its `~/.ssh/authorized_keys` file to verify the signature.
5. **Access:** If they match, a secure, encrypted tunnel is opened.

Harbor Master

admin

MARIADB ENGINE VERSION *

MariaDB 10.11

Logout

Cluster Infrastructure

GALERA-NODE-1 (IP) *
192.168.64.2

NODE-NAME-1 *
db-node-01

GALERA-NODE-2 (IP) *
192.168.64.3

NODE-NAME-2 *
db-node-02

GALERA-NODE-3 (IP) *
192.168.64.4

NODE-NAME-3 *
db-node-03

LVS-1 IP *
192.168.64.5

LVS-2 IP *
192.168.64.6

ASYNC-NODE IP *
192.168.64.7

MONITOR-NODE IP *
192.168.64.8

VIRTUAL-IP (VIP) *
192.168.64.150

SST Security

WSREP_SST_METHOD *
mariabackup

WSREP_SST_AUTH (USER:PASS) *
••••••••••••••••••••

Replication

REPL_USER *
repl_user

REPL_PASSWORD *
••••••••••••••••••••

Cluster Identity

WSREP_CLUSTER_NAME *
Galera_Cluster

WSREP_PROVIDER *
/usr/lib/galera/libgalera_smm.so

Version Specific Parameters: 10.11

WSREP_MODE
REQUIRED_PRIMARY_KEY,STRICT_REPLICATION

BINLOG_EXPIRE_LOGS_SECONDS
604800

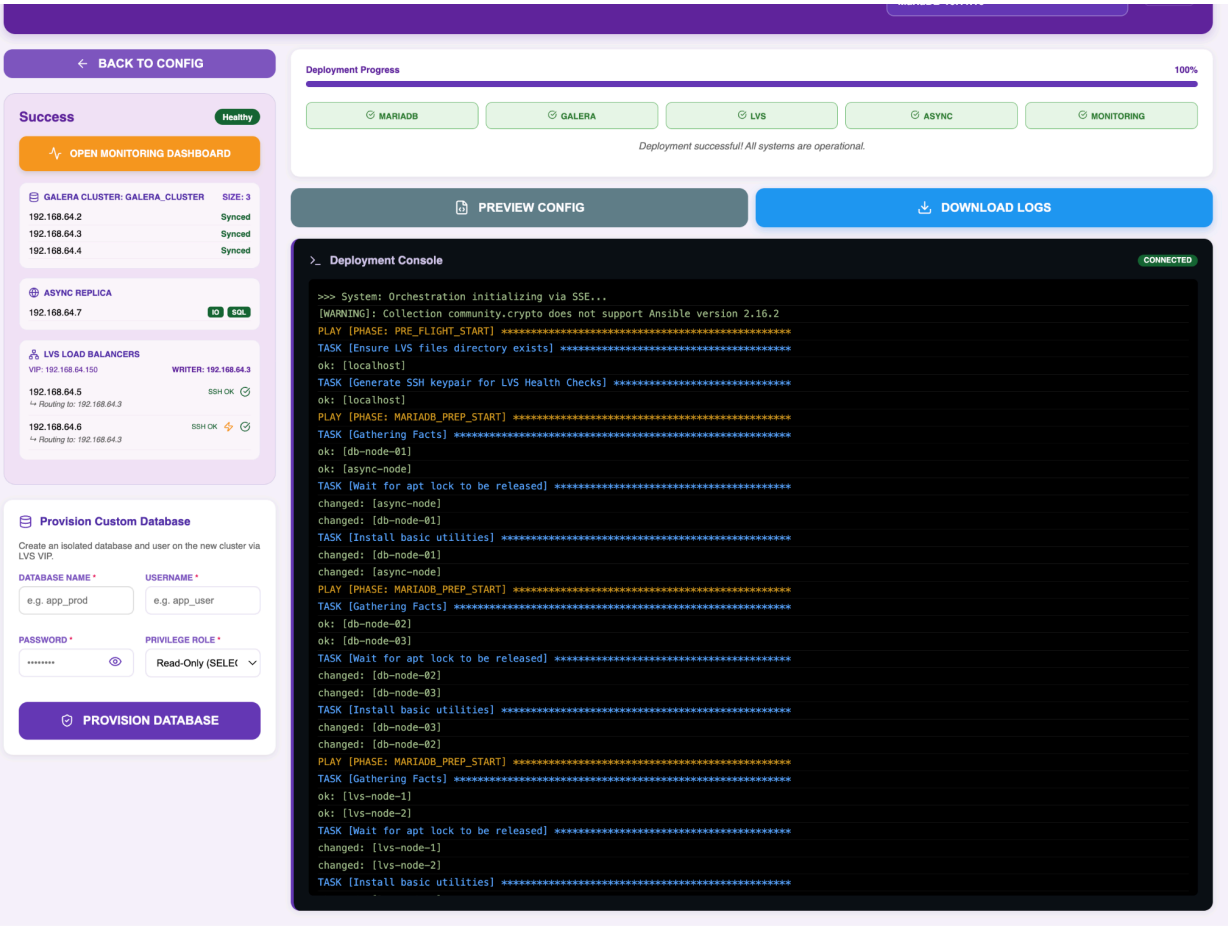
INNODB_BUFFER_POOL_INSTANCES
1

WSREP_SLAVE_THREADS
4

INNODB_UNDO_TABLESPACES
3

WSREP_GTID_DOMAIN_ID
1

VERIFY & PROCEED


BACK TO CONFIG
Success
Healthy
OPEN MONITORING DASHBOARD
GALERA CLUSTER: GALERA_CLUSTER
SIZE: 3
192.168.64.2
Synced
192.168.64.3
Synced
192.168.64.4
Synced
ASYNC REPLICA
192.168.64.7
IO
SQL
LVS LOAD BALANCERS
VIP: 192.168.64.150
WRITER: 192.168.64.3
192.168.64.5
SSH OK
Routing to: 192.168.64.3
192.168.64.6
SSH OK
Routing to: 192.168.64.3
Provision Custom Database
Create an isolated database and user on the new cluster via LVS VIP.
DATABASE NAME *
e.g. app_prod
USERNAME *
e.g. app_user
PASSWORD *
PRIVILEGE ROLE *
Read-Only (SELE
PROVISION DATABASE
Deployment Progress
100%
MARIADB
GALERA
LVS
ASYNC
MONITORING
Deployment successful! All systems are operational.
PREVIEW CONFIG
DOWNLOAD LOGS
Deployment Console
CONNECTED
>>> System: Orchestration initializing via SSE...
[WARNING]: Collection community.crypto does not support Ansible version 2.16.2
PLAY [PHASE: PRE_FLIGHT_START] ***********
TASK [Ensure LVS files directory exists] ***********
ok: [localhost]
TASK [Generate SSH keypair for LVS Health Checks] ***********
ok: [localhost]
PLAY [PHASE: MARIADB_PREP_START] ***********
TASK [Gathering Facts] ***********
ok: [db-node-01]
ok: [async-node]
TASK [Wait for apt lock to be released] ***********
changed: [async-node]
changed: [db-node-01]
TASK [Install basic utilities] ***********
changed: [db-node-01]
changed: [async-node]
PLAY [PHASE: MARIADB_PREP_START] ***********
TASK [Gathering Facts] ***********
ok: [db-node-02]
ok: [db-node-03]
TASK [Wait for apt lock to be released] ***********
changed: [db-node-02]
changed: [db-node-03]
TASK [Install basic utilities] ***********
changed: [db-node-03]
changed: [db-node-02]
PLAY [PHASE: MARIADB_PREP_START] ***********
TASK [Gathering Facts] ***********
ok: [lvs-node-1]
ok: [lvs-node-2]
TASK [Wait for apt lock to be released] ***********
changed: [lvs-node-1]
changed: [lvs-node-2]
TASK [Install basic utilities] ***********

Success

Healthy

OPEN MONITORING DASHBOARD

GALERA CLUSTER: GALERA_CLUSTER

192.168.64.182	Size: 3	Synced
192.168.64.183	Size: 3	Synced
192.168.64.184	Size: 3	Synced

ASYNC REPLICA

192.168.64.187	IO	SQL

LVS LOAD BALANCERS

VIP: 192.168.64.200

192.168.64.185	[VIP HOLDER]
192.168.64.186	

Provision Custom Database

Create an isolated database and user on the new cluster via LVS VIP.

DATABASE NAME *

e.g. app_prod

USERNAME *

e.g. app_user

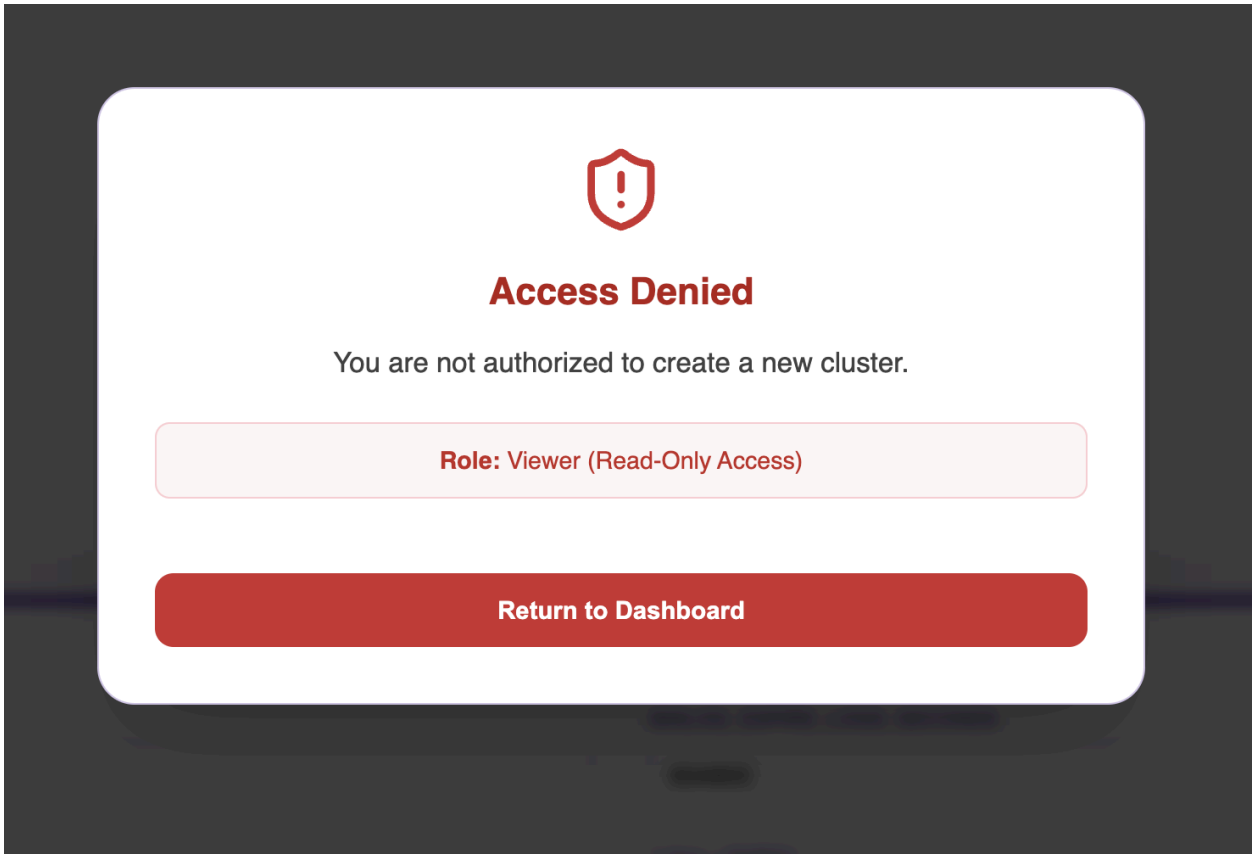
PASSWORD *

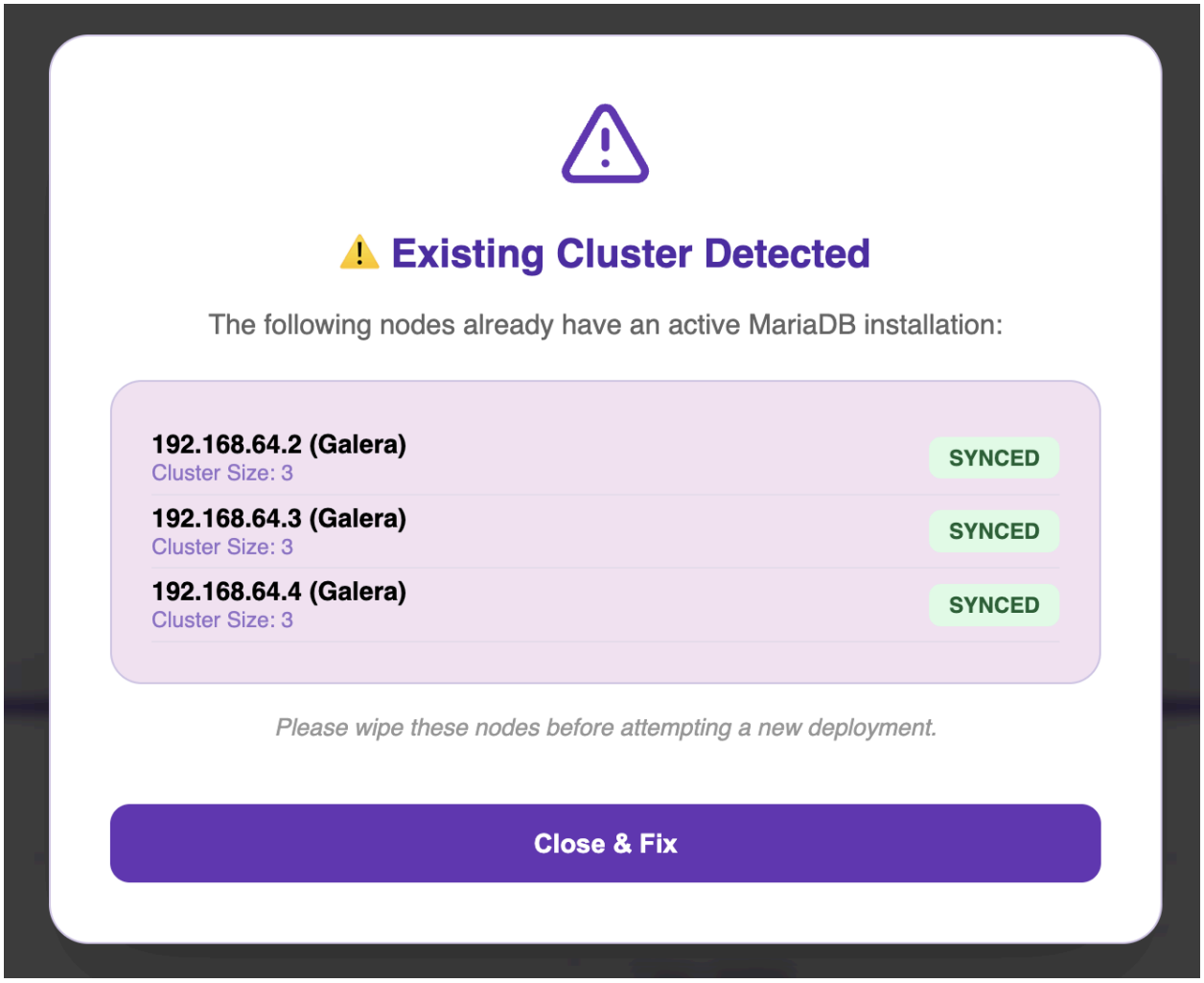
••••••••

PRIVILEGE ROLE *

Read-Only (SELEC

PROVISION DATABASE


Access Denied
You are not authorized to create a new cluster.
Role: Viewer (Read-Only Access)
Return to Dashboard


⚠️ Existing Cluster Detected
The following nodes already have an active MariaDB installation:
192.168.64.2 (Galera)
Cluster Size: 3
SYNCED
192.168.64.3 (Galera)
Cluster Size: 3
SYNCED
192.168.64.4 (Galera)
Cluster Size: 3
SYNCED
Please wipe these nodes before attempting a new deployment.
Close & Fix

Testing Commands -

Galera and Async

sudo mariadb -u root -p'AdminSecurePass123!' -e "SHOW STATUS LIKE 'wsrep_cluster_size'; SHOW STATUS LIKE 'wsrep_local_state_comment'; SHOW STATUS LIKE 'wsrep_connected';"

sudo mysql -e "SHOW SLAVE STATUS\G"

Database Creation and replication

On db-01 -

mysql -u root -p

CREATE DATABASE DBPROD;

SELECT User, Host, JSON_VALUE(Priv, '$.password_lifetime') AS ttl_days, FROM_UNIXTIME(JSON_VALUE(Priv, '$.password_last_changed')) AS last_changed FROM mysql.global_priv WHERE User = 'ttl_admin';

On async -

sudo mysql

SHOW DATABASES;

LVS -

sudo ipvsadm -Ln

Dummy 0 on db-01 check

ip addr show dummy0

Shifting of dummy 0 Ip -

while true; do mariadb -h 192.168.64.150 -u root -p'AdminSecurePass123!' -sN -e "SELECT concat('Connected to: ', @@hostname, ' | Cluster Size: ', (SELECT VARIABLE_VALUE FROM information_schema.global_status WHERE VARIABLE_NAME='wsrep_cluster_size'))" 2>/dev/null || echo "Searching for Writer..."
sleep 1
done

Generated keys
Added vip to the galera loop back add
Create dummy 0 interface
Installed keepalived in 2 galera nodes
Shifted files
{ src: 'galera_hcip_manager.sh.j2', dest: '/usr/local/sbin/galera_healthcheck.sh', mode: '0755' }
{ src: 'db_keepalived.conf.j2', dest: '/etc/keepalived/keepalived.conf', mode: '0644' }

On Galera
Db.keepalived.cnf runs every few sec the galera_hcip.manager.sh, this particular script sees if node is synced or not and returns 0 for success and 1 for failure, on success db.keepalived.cnf assigns hcip value to the dummy 0 interface
On LVS keepalived.conf.j2 uses MISC_CHECK which triggers the check_hcip_ssh.sh , this file checks via ssh if hcip is present on its dummy0 interface , if success the keepalived.conf.j2 will add db node to the traffic taking one.

Other validation Checks

- [ ] **RAM Check:** Calculate if innodb_buffer_pool_size $\le$ **80%** of ansible_memtotal_mb. - To ensure a 'zero-touch' stable environment for a non-technical user, we should provision **8GB RAM per node**. This allows us to allocate a generous 5GB for data caching while still leaving a 3GB buffer to handle Galera replication overhead and OS tasks, effectively preventing Out-Of-Memory (OOM) crashes during cluster synchronization.
- [ ] **CPU Check:** Verify wsrep_slave_threads matches the detected ansible_processor_vcpus.
- [ ] **Disk Headroom:** Check ansible_mounts to ensure /var/lib/mysql (or the root partition) has at least **10-20GB** (or $2\times$ the existing DB size) of free space for SST.

Config Parameters Details:

```
wsrep_on                = ON
wsrep_provider          = /usr/lib/galera/libgalera_smm.so
binlog_format           = ROW
default_storage_engine  = InnoDB
innodb_autoinc_lock_mode = 2
bind-address            = 0.0.0.0
```

```
# --- Cluster Identity (Required) –

wsrep_cluster_name       = "Galera_Cluster"
wsrep_cluster_address    = "gcomm://192.168.1.10,192.168.1.11,192.168.1.12"

# --- Node Identity (Unique to this Node) –  For every Node
wsrep_node_address       = 192.168.1.11
wsrep_node_name          = node2
server_id                = 102

# --- SST / Sync (Required) ---
wsrep_sst_method         = mariabackup
wsrep_sst_auth           = sst_user:password

# --- 10.5 Specific / Optional ---
wsrep_strict_ddl         = ON              # Replicated strictly
wsrep_replicate_myisam   = OFF             # Experimental in 10.5
expire_logs_days         = 7               # Legacy log rotation

# --- 10.6 Specific / Optional ---
wsrep_mode               = REQUIRED_PRIMARY_KEY,STRICT_REPLICATION
binlog_expire_logs_seconds = 604800        # Replaces expire_logs_days
innodb_buffer_pool_instances = 1           # Optimized for 10.6+

# --- 10.11 Specific / Optional ---
wsrep_mode               = REQUIRED_PRIMARY_KEY,STRICT_REPLICATION
binlog_expire_logs_seconds = 604800
wsrep_slave_threads      = 4               # Recommended to speed up applier
innodb_undo_tablespaces  = 3               # Default in 10.11 for better I/O
wsrep_gtid_domain_id     = 1               # Recommended for cluster GTID isolation
```

Galera Config definitions:

wsrep_on = ON
By default its off but when we turn this on we are making the node part of a cluster and asking it to keep data in sync and search its neighbour and it should replicate
We can however turn this off, so our node becomes lonely and will not broadcast its data which is useful in debugging

wsrep_provider = /usr/lib/galera/libgalera_smm.so
This is the engine that stores the logic for replication. It points to a specific file in our hard drive, the .so file . This file handles group communication, make sure every node is on the same page and conflict resolution

The file (libgalera_smm.so) is created on your hard drive when you install the Galera software package.
Changes on the basis on operating system
Debian / Ubuntu - /usr/lib/libgalera_smm.so
Red Hat / CentOS / AlmaLinux - /usr/lib64/libgalera_smm.so
By default, this is empty. This is why standard MariaDB installations don't start as clusters automatically; you have to manually tell it where the Galera library lives.

binlog_format = ROW

binlog_format determines how the database records changes in its binary logs. There are three possible formats: Statement, Mixed, and Row. Galera only works reliably with Row.To understand why Galera insists on ROW, you have to look at how the other formats work:
Statement-Based (STATEMENT): The database logs the literal SQL command you typed (e.g., UPDATE users SET salary = salary * 1.1 WHERE department = 'Sales').
The Problem: If Node A and Node B have slightly different data, or if the query uses a non-deterministic function like NOW() or RAND(), the result on Node B might be different from Node A.

Row-Based (ROW): The database logs exactly which bits of data actually changed on the disk. Instead of the "Sales" command, it logs: "In row #502, change salary from 5000 to 5500."
The Benefit: This is 100% predictable. It ensures that every node in the cluster ends up with the exact same data, down to the last byte.

What is the Binlog Position?

The binlog position consists of two parts:

1. The Log File: The name of the current file (e.g., mariadb-bin.000001).
2. The Position: A numeric offset inside that file (e.g., 542).

default_storage_engine = InnoDB
Galera is built to work with InnoDB. If a developer runs a CREATE TABLE command and forgets to specify an engine, this variable decides which one is used by default.

If you left this at the old MySQL default of MyISAM, a user could create a table that doesn't replicate across your cluster without even realizing it.With default_storage_engine = InnoDB: Every new table automatically gets the "Galera-compatible" engine. It gets row-level locking, transaction support, and (most importantly) it stays in sync across all your nodes.

innodb_autoinc_lock_mode = 2

It changes how the database handles AUTO_INCREMENT values when multiple connections are trying to insert data at the same time.There are three modes (0, 1, and 2). Setting this to 2 is known as "Interleaved" mode.

When data arrives at the first galera node, the node has to tell the other 2 galera nodes that its providing a sequence number to it. If suppose 100 request arrive at node 1, the node has to make other nodes wait to longer period of time. With the auto increment as 2 we are proivding every galera node a new lane of sequence number so they are work independently and don't clash.

Mode 1 (Consecutive): The database takes a "table-level lock" to ensure that if you insert 100 rows at once, they get 100 perfectly consecutive numbers (1, 2, 3...).
The Galera Problem: In a multi-master cluster, if Node A and Node B both take a table-level lock at the same time, the cluster grinds to a halt while they argue over who gets the next number. This causes massive performance bottlenecks.
Mode 2 (Interleaved): The database never takes a table-level lock. It assigns numbers as fast as possible.
The Galera Benefit: It allows multiple "librarians" (the wsrep_slave_threads we discussed) to write data in parallel without stopping to wait for a lock. This is the only mode that truly supports high-speed, parallel replication.

How it works: Each store (Node) gets its own "lane."

- Node A only hands out numbers like: 1, 4, 7, 10...
- Node B only hands out numbers like: 2, 5, 8, 11...
- Node C only hands out numbers like: 3, 6, 9, 12...

Result: No one has to call anyone else! Node A can hand out 100 tickets in a second without ever checking what Node B is doing.

bind-address = 0.0.0.0

We set bind-address to 0.0.0.0 to ensure the database listens on all available network interfaces. This is necessary for Galera Cluster nodes to communicate with each other across the network, rather than being restricted to local-only connections."
--- Cluster Identity (Required) —

wsrep_cluster_name = "Galera_Cluster"
It's a unique Identifier for the group of servers. Its primary job is to prevent accidental connection. It is case sensitive and also has a default value and you should never leave this part as default. You can change the cluster name but you should never do that because it allows the server to leave the cluster and find a new connection with that name.
wsrep_cluster_address = "gcomm://192.168.1.10,192.168.1.11,192.168.1.12"
It's like a phone book that tells the node where to find its teammates. Its uses gcomm meaning group communication which is a special galera based internal networking logic

It is best practice to list **all** nodes in the cluster, separated by commas
standard MySQL/MariaDB port (3306)
It uses port 4567 for cluster communication
Its should be left empty for the second third node as it will make the cluster think as the primary one and will lead to a split brain scenario
4568 IST port
4444 SST port

--- Node Identity (Unique to this Node) — For every Node
wsrep_node_address = 192.168.1.11
It's the node's own phonebook. If you don't set wsrep_node_address, the node will try to guess what its address is, which often leads to the node "joining" the cluster logically but never actually receiving any data because the other nodes are sending it to the wrong IP.

wsrep_node_name = node2
Humans use node names to make clusters easier to manage. The primary reason to give your nodes friendly names is for **State Snapshot Transfers (SST)**. When a new node joins the cluster, it needs to copy all the data from an existing member (the "Donor").

Instead of trying to remember which IP belongs to which server, you can use the node name. We can change the name and its dynamic and can come into effect without causing any damage. Unlike a primary key in a database, Galera does **not** force wsrep_node_name to be unique. Multiple nodes can have the same name. **When a new node joins the cluster, galera provides a unique UUID and its ip address to distinguish between the nodes on the cluster.**

server_id = 102
It's like a secondary badge. It's basically used to prevent cluster replication
Even though Galera is doing the "heavy lifting," if you leave the server_id blank or set it to 0, many versions of the database will refuse to replicate properly or will run into issues with the Binary Log.0 is a special value that usually means "No ID," which effectively disables replication. Avoid using it in a cluster.

--- SST / Sync (Required) ---
wsrep_sst_method = mariabackup
When a node starts up it realizes it has no data and is out of sync so galera chooses a donor and once the donor is chosen it uses one of the methods defined in this variable to do the sst backup.

How exactly it works
rsync (Default) - Uses the Linux rsync tool to copy files. Extremely fast and reliable.**Blocks the Donor:** The node sending the data becomes unresponsive to the app until the transfer is done.

Mariadb-backup - Uses MariaDB's native backup tool.**Non-blocking:** The donor node stays online and can still read/write data while sending.Slightly more complex to set up (requires a specific database user).
xtrabackup-v2-Uses Percona's XtraBackup tool.Also non-blocking. Very popular for large datasets.Requires installing external software (percona-xtrabackup).
Mysqldump - Generates a massive SQL script.Works on any system.**Very slow.** Not recommended for modern clusters or large databases.
Every node in the cluster **must have the same** **wsrep_sst_method**.

wsrep_sst_auth = sst_user:password

This is the security check in for the backup. **rsync** works at the File System level (it copies the raw data files directly from the disk), so it doesn't need to log into the database. So its not required for rsync. **mariadb-backup** and **xtrabackup** work at the Database level. They need to connect to the donor node, run commands to prepare the data, and manage logs. To do this, they need a username and password.
We have created a user in the first database . Because an SST transfer is a powerful operation, the user needs specific "administrative" privileges.
If you try to run a command and see the password it will be masked.

 # --- 10.5 Specific / Optional ---
wsrep_strict_ddl = ON # Replicated strictly
By default mariadb allows you to create MYISAM tables in a cluster. But there are chances that if you write data to a MyISAM table on Node A, it might **not** show up on Node B. This creates a "split" in your data that can lead to massive headaches later.
With this specific field ON if you try to run a CREATE TABLE or ALTER TABLE command using an unsupported engine (like MyISAM), the database will reject the command and throw an error.
It forces you (or your developers) to use InnoDB, ensuring that every table in your database is actually being replicated to all nodes.

wsrep_replicate_myisam = OFF # Experimental in 10.5
When set to **ON**, Galera would attempt to "broadcast" INSERT, UPDATE, and DELETE commands (DML) performed on MyISAM tables to the other nodes.
However, because MyISAM doesn't support the same locking mechanisms as InnoDB, there was no guarantee that the data would land safely or in the right order if multiple nodes were writing to the same MyISAM table at once. It was the "**Experimental Bridge**" that tried to make MyISAM work within a Galera Cluster.

expire_logs_days = 7 # Legacy log rotation
Used for cleaning up of binlogs. This variable tells mariadb to clean up the logs which are older than a week. The cleanup happens when the server starts or when a log file "rolls over". Unit: Days.
 # --- 10.6 Specific / Optional –

wsrep_mode = REQUIRED_PRIMARY_KEY,STRICT_REPLICATION

Instead of having ten different variables to toggle, MariaDB now uses this single "Enumeration" (a list of flags) to define exactly how strict or experimental your cluster should be.
This replaces the old wsrep_strict_ddl. It ensures that you cannot create or alter tables unless they use the **InnoDB** engine.
Galera relies on Primary Keys to identify which row is being changed across the cluster. Without a Primary Key, Galera has to use "Full Table Scans" to find the right row, which is incredibly slow and prone to conflicts. Turning this on forces developers to include a Primary Key in every CREATE TABLE statement.
Global Transaction IDs (GTIDs) are how databases track their position in a timeline. This flag is a "Safety Lock." It prevents the node from generating "local" transaction IDs that only exist on one machine. If you try to do something that would create a local-only record (like writing to a MyISAM table or turning wsrep_on OFF), the database will simply block the command with an error.
Galera **requires** binlog_format=ROW. If someone accidentally tries to change the log format to STATEMENT or MIXED (which would break Galera), this mode will prevent that change from happening.

binlog_expire_logs_seconds = 604800 # Replaces expire_logs_days
modern successor to the expire_logs_days. In modern high-speed databases, a single "Day" can produce **Terabytes** of logs. If your disk has 500GB of space and you generate 100GB of logs every 3 hours, you will crash before the first "Day" is over. With **seconds**, you can set the expiration to 10800 (3 hours), keeping your disk safe.

innodb_buffer_pool_instances = 1 # Optimized for 10.6+
Buffer Pool is the most important piece of memory—it's where InnoDB caches your data and indexes so it doesn't have to go to the slow hard drive.innodb_buffer_pool_instances is the variable that decides whether that memory should be one giant "pool" or divided into multiple "smaller pools." Default value is 1.

--- 10.11 Specific / Optional —

wsrep_mode = REQUIRED_PRIMARY_KEY,STRICT_REPLICATION
binlog_expire_logs_seconds = 604800

wsrep_slave_threads = 4 # Recommended to speed up applier
In a Galera Cluster, when a "Write Set" (data update) arrives from another node, this variable determines how many threads can work at the same time to write that data to the local disk.
The Problem: If Node A is sending 500 updates per second, but a single thread on Node B can only process 200 per second, Node B will start to lag.
The Solution: Increasing wsrep_slave_threads allows Galera to multi-task. It looks at the incoming data and says, *"Update 1 and Update 2 don't touch the same rows, so Librarian A and Librarian B can write them at the exact same time."*

*A common "Rule of Thumb" is to match the number of **CPU cores** on your server.*

- *If you have a 4-core CPU, try `wsrep_slave_threads = 4`.*
- *If you have a massive 32-core server, you might go up to `16` or `32`.*

> ***Note:** Setting this higher than your CPU core count usually provides "diminishing returns." If you have 2 cores but 512 threads, the threads will just spend all their time fighting over the CPU.*

innodb_undo_tablespaces = 3 # Default in 10.11 for better I/O
Whenever a transaction is running, InnoDB needs a place to store the "old" version of the data in case you decide to `ROLLBACK` or if another user needs to see the data as it existed before you started changing it (this is called Multi-Version Concurrency Control, or MVCC).`innodb_undo_tablespaces` determines how many physical files on your disk are dedicated to holding this "undo" data.While the minimum required for automatic shrinking is **2**, setting it to **3** (or more) provides a "spare.

wsrep_gtid_domain_id = 1 # Recommended for cluster GTID isolation

MariaDB keeps track of time across different servers using **GTIDs (Global Transaction IDs)**.A GTID looks like this: `0-1-100`. The first number (`0`) is the **Domain ID**.

`0-1-152` (Domain ID - Server ID - Sequence Number)

Usually, MariaDB uses `gtid_domain_id`. However, Galera is a "Multi-Master" system—anyone can write at any time.

If you set `wsrep_gtid_mode = ON`, the cluster stops using the standard `gtid_domain_id` and starts using `wsrep_gtid_domain_id`. This ensures that every node in the cluster produces a **consistent, identical GTID** for the same transaction.

Without this, Node A might label a change as `1-1-50` and Node B might label the exact same change as `2-1-50`. That would break any standard "Master-Slave" replicas connected to the cluster.

Same for All Nodes: Every node in the same Galera Cluster **must** have the exact same `wsrep_gtid_domain_id`. If they differ, the GTID sequence will become a mess.
Unique Across Clusters: If you are replicating data *between* two different clusters, Cluster A should be `1` and Cluster B should be

The Domain ID is the **Grouping Mechanism**. It tells MariaDB: *"These transactions belong to this specific category of data, and they can be processed independently of other categories."*

MariaDB Enterprise Cluster requires that you set a name for your cluster, using the wsrep_cluster_name system variable. When nodes connect to each other, they check the cluster name to ensure that they've connected to the correct cluster before replicating data. All Servers in the cluster must have the same value for this system variable.

Using the wsrep_cluster_address system variable, you can define the back-end protocol (always gcomm) and a comma-separated list of the IP addresses or domain names of the other nodes in the cluster.

It is best practice to list all nodes on this system variable, as this is the list the node searches when attempting to reestablish network connectivity with the primary component.

To achieve true Active-Active load balancing on a single Virtual IP, we would transition from VRRP-based failover to ECMP (Equal-Cost Multi-Path) routing. By using a routing daemon like FRR integrated with Keepalived, both LVS nodes would simultaneously advertise the same VIP to our upstream router via BGP. This allows the network hardware itself to distribute traffic across both load balancers at wire speed, providing massive scalability without ever needing to change the application's connection string.

Ansible Keywords:

Ansible facts are system-level data (OS, network, hardware) automatically collected from managed nodes into the `ansible_facts` variable, enabling dynamic, conditional playbooks. Gathered via the setup module, this data is crucial for targeting specific configurations, such as installing packages based on the OS family.